

# JARVIS

## Local-First Personal AI Assistant for Windows

Comprehensive Technical Architecture, Subsystem Engineering & End-to-End Codebase Reference. Designed for private, local execution with cloud fallbacks, hybrid vector search, tool execution sandbox, and push-to-talk voice pipeline.

**Author:** Engineering Team • **Version:** 1.0.0 Production • **Date:** September 2026

# Table of Contents

---

- 1. Executive Summary & Design Philosophy** — Local-first principles, privacy boundaries, graceful degradation
- 2. System Architecture & High-Level Topology** — Subsystem boundaries, process model, and threading hierarchy
- 3. Application Lifecycle & Startup Sequence** — Two-process architecture, bootstrap flow, 8-point health checks
- 4. Configuration & Settings Management** — Zero magic numbers, Pydantic BaseSettings, dynamic path derivation
- 5. LLM Provider Layer & Fallback Chains** — Protocol abstraction, Groq exponential backoff, rate-limit parsing, Ollama translation
- 6. LangGraph Agent Orchestration** — State machine, iterative tool loop, provider pinning, confirmation interruption
- 7. Tool System & Sandboxed Execution** — Tool registry, JSON Schema generation, risk tiering, deterministic confirmation
- 8. Local RAG & Hybrid Retrieval Pipeline** — File crawling, SHA-256 change detection, token chunking, FAISS + FTS5 BM25, cross-encoder reranking
- 9. SQLite Storage Layer & Schema Migrations** — WAL mode, connection pooling, 12 tables, idempotent migration runner, repositories
- 10. Web Search & Live Page Extraction** — SearXNG JSON client, lazy subprocess management, crawler TTL cache, Readability extraction
- 11. Voice Subsystem (Local & Cloud Hybrid)** — PortAudio streaming, RMS silence gate, STT & TTS 3-stage fallback chains
- 12. Security, Secret Masking & Audit Logging** — Secret scrubbing, non-raising audit logging, prompt injection defense
- 13. PySide6 Desktop User Interface** — Responsive QThread workers, Markdown bubbles, interactive citations, animated presence dot
- 14. Windows OS Integration** — Task Scheduler logon task, global hotkey, allowlisted app launch, workstation lock
- 15. Document Parsing Architecture** — Unified parser protocol, PyMuPDF, python-docx, python-pptx, openpyxl, text
- 16. Background Scheduler & Reminders** — APScheduler service, background polling, Qt toast notifications
- 17. End-to-End Execution Trace** — Complete step-by-step trace of a real multi-tool research turn
- 18. Exhaustive File-by-File Code Reference** — Detailed breakdown of every single source file in the repository

# 1. Executive Summary & Design Philosophy

**JARVIS** is a local-first, privacy-respecting personal AI assistant engineered specifically for Windows desktop environments. Unlike cloud-dependent digital assistants that transmit user activity, notes, and documents to third-party data centers, JARVIS is built from the ground up to retain user sovereignty. All long-term memories, notes, tasks, reminder schedules, and document indexes reside exclusively in a local SQLite database (`data/jarvis.db`) and local FAISS vector indices.

## Core Architectural Pillars

- **Local-First Data Ownership:** No telemetries, no centralized user telemetry, and no synchronization clouds. The system functions fully offline for local document indexing, personal notes/tasks, local STT (faster-whisper), and local TTS (Piper). Cloud services are used strictly for accelerated LLM inference (Groq/Ollama Cloud) or web discovery (SearXNG).
- **Zero Secret Leakage:** User API keys (Groq, Deepgram, Ollama) exist solely in the local `.env` file. The application enforces a hard security boundary via `security/secrets.py`, ensuring that raw keys are never passed to LLM context windows, never persisted into the database, and masked in all application log files.
- **Human-in-the-Loop Risk Gating:** Tools are categorized into risk tiers (LOW, MEDIUM, HIGH). Dangerous operations (such as modifying/writing local files, deleting database records, launching desktop applications, or locking the workstation) are halted immediately at the agent level. Execution pauses and requires explicit user confirmation via the desktop UI before proceeding.
- **Resilient Graceful Degradation:** No peripheral failure crashes the core chat assistant. If SearXNG is unavailable, web search returns a graceful error without breaking the conversation; if audio hardware is absent, voice features hide automatically; if primary LLM providers hit rate limits, the manager seamlessly fails over to backup providers.
- **Strict Elimination of Magic Numbers:** All timing thresholds, retry counts, token budgets, retrieval weights, chunk dimensions, and UI dimensions are declared as documented constants in `config/defaults.py`, overridable through environment variables via `config/settings.py`.

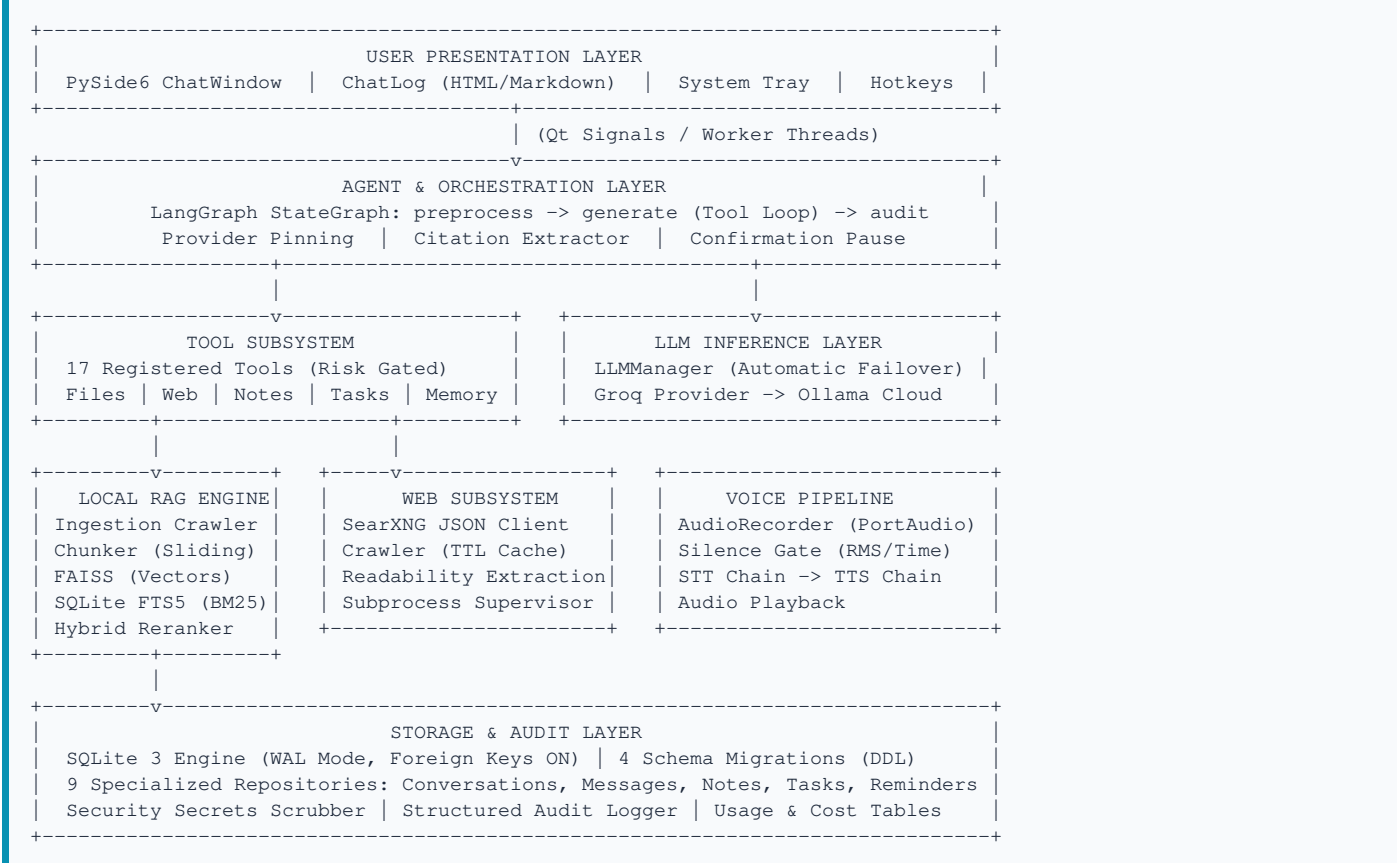
## Technology Matrix

| Subsystem           | Primary Implementation                   | Fallback / Alternative                 | Location                                  |
|---------------------|------------------------------------------|----------------------------------------|-------------------------------------------|
| Runtime & Framework | Python 3.12+ (64-bit AMD64)              | PySide6 (Qt 6.6+) Desktop GUI          | <code>app/ui/</code>                      |
| Agent Orchestration | LangGraph StateGraph                     | Deterministic state machine loop       | <code>agent/graph.py</code>               |
| Primary LLM         | Groq (openai/gpt-oss-120b)               | OpenAI-compatible HTTP client          | <code>llm/groq_provider.py</code>         |
| Fallback LLM        | Ollama Cloud (gpt-oss:120b)              | Native /api/chat payload translation   | <code>llm/ollama_cloud_provider.py</code> |
| Database Storage    | SQLite 3 with WAL journal mode           | FTS5 full-text keyword indexing        | <code>storage/database.py</code>          |
| Embeddings          | sentence-transformers (all-MiniLM-L6-v2) | Local CPU inference (384 dims)         | <code>rag/embeddings.py</code>            |
| Vector Index        | FAISS CPU (IndexFlatL2)                  | Local binary index serialization       | <code>rag/vector_store.py</code>          |
| Reranker            | cross-encoder/ms-marco-MiniLM-L-6-v2     | Local sequence classification          | <code>rag/reranker.py</code>              |
| Web Search          | SearXNG (Self-Hosted JSON API)           | Optional auto-spawned subprocess       | <code>web/search.py</code>                |
| Web Extraction      | readability-lxml (Mozilla Readability)   | BeautifulSoup HTML tag stripper        | <code>web/extraction.py</code>            |
| Speech-to-Text      | Groq Whisper Large v3 Turbo              | Deepgram Nova-3 → faster-whisper local | <code>voice/stt*.py</code>                |
| Text-to-Speech      | Groq PlayAI Orpheus                      | Deepgram Aura → Piper local ONNX       | <code>voice/tts*.py</code>                |
| OS Integration      | Task Scheduler (schtasks.exe)            | ctypes Win32 API, keyboard hook        | <code>app/windows_startup.py</code>       |

## 2. System Architecture & High-Level Topology

JARVIS employs a layered, modular architecture with unidirectional dependencies where possible. The presentation layer (PySide6) communicates with the cognitive layer (LangGraph Agent) via non-blocking asynchronous worker threads. The cognitive layer coordinates the tool subsystem, which in turn queries the local RAG engine, storage repositories, web services, and Windows system APIs.

### Subsystem Layer Hierarchy



### Threading & Concurrency Model

To guarantee a smooth, stutter-free 60 FPS user interface, JARVIS strictly segregates tasks across dedicated execution contexts:

- Main GUI Thread:** Runs the PySide6 event loop, renders message bubbles, animates the presence indicator dot, handles user input events, and triggers notification modals. Never executes network I/O, database writes, or model inference.
- Agent Execution Thread (`chatTurnWorker`):** A `QThread` executing the complete LangGraph tool loop, handling LLM network calls and tool executions in a dedicated Python `asyncio` event loop.
- Confirmation Execution Thread (`confirmTurnWorker`):** Runs when a user approves a HIGH-risk action, deterministically executing the pending tool handler off the main thread.
- Voice Processing Thread (`voiceTurnWorker`):** Manages the end-to-end voice loop: audio buffer reading, STT network call, agent execution, TTS audio synthesis, and soundcard output.
- Audio Input Callback Thread:** PortAudio OS-level background thread capturing raw PCM float32 samples from the default microphone into an in-memory ring buffer.
- Scheduler Polling Thread:** Managed by APScheduler's `BackgroundScheduler`, running every 30 seconds to query due reminders and dispatch Qt signals to the UI.

## 3. Application Lifecycle & Startup Sequence

### The Two-Process Architecture

Desktop AI applications frequently suffer from high idle memory consumption. JARVIS solves this through an innovative **Two-Process Architecture**:

- **Supervisor Process (`app/supervisor.py`):** An ultralight background daemon consuming less than 3 MB of RAM. It registers the system-wide global hotkey (Ctrl+Space) using a low-level keyboard hook and maintains a lightweight system tray icon. It imports zero heavy machine learning libraries (no PyTorch, FAISS, LangGraph, or sentence-transformers). When the hotkey is pressed, it inspects running Windows processes using Win32 `FindWindowW`. If JARVIS is already open, it brings the window to the foreground via `SetForegroundWindow`; if not, it spawns the full application via `start_jarvis.bat` with an 8-second debounce guard.
- **Main Application Process (`app/main.py`):** The comprehensive assistant process containing all cognitive models, GUI windows, and database pools (~150 MB RAM). When the user closes the chat window or clicks "Quit", this process terminates completely, freeing all memory while leaving the 3 MB supervisor active.

### Bootstrap Sequence

When `app/main.py` executes, it invokes `app/bootstrap.py:bootstrap()`, executing the following ordered sequence:

1. **Environment & Settings Resolution:** Instantiates the singleton `Settings` object via Pydantic, parsing `.env` and computing derived filesystem paths.
2. **Structured Logging Initialization:** Sets up dual-destination logging (console standard output and rotating log file at `data/logs/jarvis.log`) with log-level controls.
3. **Storage Initialization:** Establishes the SQLite connection pool, enabling WAL mode, setting synchronous to NORMAL, and activating foreign key enforcement.
4. **Schema Migration Execution:** Compares the database state against the version ledger in `schema_migrations` and sequentially applies unexecuted DDL scripts.
5. **SearXNG Process Manager Acquisition:** Instantiates the singleton `SearXNGProcessManager` in passive mode (lazy launch postponed until the first search).
6. **Startup Audit Logging:** Writes an entry into the `audit_log` table with host information, timestamp, and version metadata.
7. **Comprehensive Health Checks:** Executes `app/lifecycle.py:run_health_checks()` across 8 core subsystems (Configuration, Database, Vector Store, LLM Connectivity, Web Search, Voice Audio, Scheduler, and UI subsystem). Health warnings are aggregated into a non-intrusive status banner in the chat UI.
8. **UI Presentation:** Instantiates `ChatWindow`, connects background workers, registers signal slots, and displays the interface.

## 4. Configuration & Settings Management

JARVIS strictly isolates configuration into two distinct layers:

1. `config/defaults.py` (**Default Values & System Constants**): Contains over 60 documented constant declarations defining timeout durations, retry counts, sliding window sizes, token chunk budgets, vector embedding dimensions, similarity score thresholds, hybrid retrieval weights, and security allowlists.
2. `config/settings.py` (**Dynamic Pydantic BaseSettings**): Loads user secrets, custom endpoint URLs, and feature toggles from `.env`. Employs caching via `@lru_cache` so repeated calls to `get_settings()` return the identical validated singleton.

### Dynamic Directory Derivation

All runtime data directories are dynamically derived from a single configurable root (`JARVIS_DATA_DIR`, defaulting to `./data`):

- `db_path`: `data/jarvis.db` (SQLite database)
- `logs_dir`: `data/logs` (Application activity logs)
- `indexes_dir`: `data/indexes` (FAISS vector indices)
- `cache_dir`: `data/cache` (Web crawler page cache)
- `voices_dir`: `data/voices` (Piper TTS offline voice ONNX models)

## 5. LLM Provider Layer & Fallback Chains

LLM interactions are governed by a strict abstract interface (`LLMProvider` Protocol in `llm/base.py`). Every provider accepts standardized message dictionaries and tool definitions, and returns a strongly-typed `LLMResponse`:

```
class LLMResponse(BaseModel):
    content: str | None
    provider: str
    model: str
    finish_reason: str
    tool_calls: list[dict[str, Any]]
    raw: dict[str, Any]
    usage: TokenUsage
```

### Intelligent Fallback Architecture (`LLMManager`)

Provider management is encapsulated within `llm/manager.py`. The application establishes an ordered chain of providers: **Groq (Primary) → Ollama Cloud (Fallback)**. When `LLMManager.generate()` is invoked:

- It dispatches the request to the primary provider (Groq).
- If a recoverable error occurs (network failure, timeout, invalid response, or rate limits), the manager captures the exception and immediately invokes the next provider in the chain.
- If all configured providers fail, it raises `AllProvidersFailedError` containing an itemized audit of every attempt's failure reason.

### Groq Rate-Limit Parsing & Wait-or-Abandon Strategy

Groq provides high token throughput but enforces dynamic token-bucket rate limits, returning HTTP 429 status codes accompanied by custom header formats (e.g., `retry-after: 547ms, 1.065s, or 8m38.4s`).

In `llm/groq_provider.py`, a custom parser (`parse_retry_after_header()`) extracts the wait duration. If the required wait is brief ( $\leq \text{DEFAULT\_LLM\_RATE\_LIMIT\_MAX\_WAIT\_SECONDS}$ , default 8 seconds), the provider sleeps and retries automatically. However, if the reset window exceeds 8 seconds, the provider immediately raises `ProviderRateLimitError`, triggering instant failover to Ollama Cloud without freezing the user experience.

### Ollama Cloud Tool Call Translation

Different providers adhere to varying tool call specifications. While Groq (OpenAI-compatible) expects tool call arguments serialized as a JSON string (e.g., `{"function": {"arguments": "\"\{\"query\": \"test\"}\""}}`), Ollama Cloud's native API rejects strings and demands a raw JSON dictionary object. `llm/ollama_cloud_provider.py` implements bidirectional message translation (`_translate_messages()` and `_translate_tools()`) ensuring compatibility across providers without polluting the agent state.

## 6. LangGraph Agent Orchestration

The cognitive core of JARVIS is implemented using **LangGraph** (`agent/graph.py`). The execution graph is modeled as a 3-node state machine:

```
[START] --> (preprocess) --> (generate: Tool Loop) --> (audit) --> [END]
```

### State Machine Execution Steps

- preprocess Node:** Sanitizes incoming user messages, trims whitespace, verifies conversation validity, and initializes state flags.
- generate Node (The Iterative Tool Loop):**
  - Loads recent conversation history from SQLite (capped at the last 20 messages to conserve context tokens).
  - Queries the memory repository and injects up to 30 relevant persistent facts into the system prompt.
  - Compiles the complete message array and enters the tool execution loop (max 8 iterations).
  - In each iteration, the LLM is queried with all registered tool schemas.
  - If the LLM outputs text without tool calls, the loop terminates.
  - If the LLM requests tool calls, each tool is verified against `security/permissions.py`.
  - If any requested tool is classified as **HIGH-risk**, the agent sets `requires_confirmation=True`, stores the pending request details in `confirmation_request`, and **immediately aborts the loop**.
  - If all tools are allowed, they execute asynchronously with individual timeout bounds; results are appended to the message array, and the loop continues.
- audit Node:** Extracts source citations from tool outputs, logs token consumption, writes the assistant's final message into the database, and records a structured audit event.

### Provider Pinning During Multi-Turn Tool Loops

If a turn involves multiple LLM reasoning cycles (e.g., Search → Read Webpage → Synthesize), switching providers midway through the loop can corrupt the context due to subtle schema variations. JARVIS implements **Provider Pinning**: the provider that successfully answers the initial turn query is pinned for all subsequent tool-completion iterations within that turn.

### Deterministic Confirmation Execution

When a user confirms a HIGH-risk action (such as deleting a file), the action is executed via `Agent.confirm_and_execute()`. Crucially, this method **does not re-invoke the LLM**. It executes the exact cached tool call deterministically, updates the database, and returns a verified result, preventing LLM drift or duplicate execution.



## 7. Tool System & Sandboxed Execution

Tools in JARVIS represent capability modules through which the LLM can query the local machine, search the web, or modify data. Every tool is registered in `tools/registry.py:ToolRegistry` using the `Tool` dataclass:

```
@dataclass
class Tool:
    metadata: ToolMetadata          # name, description, risk_level, requires_confirmation
    input_schema: dict[str, Any]    # JSON Schema defining required parameters
    handler: Callable[..., Coroutine[Any, Any, Any]] # Async execution function
```

### Tool Inventory & Risk Classification

| Tool Name            | Risk Tier | Confirmation? | Functionality & Security Sandbox                                                             |
|----------------------|-----------|---------------|----------------------------------------------------------------------------------------------|
| search_files         | LOW       | No            | Queries local FAISS vector and FTS5 indices for document snippets.                           |
| read_file            | LOW       | No            | Reads text from a local file. Strictly gated: path must reside within an indexed folder.     |
| list_folder          | LOW       | No            | Lists directories and files inside an explicitly indexed folder.                             |
| list_indexed_folders | LOW       | No            | Returns the list of active user-approved indexed folders.                                    |
| write_file           | HIGH      | Yes           | Writes text to disk. Gated: destination must be within an indexed folder. Requires approval. |
| web_search           | LOW       | No            | Queries local SearXNG engine for search snippets.                                            |
| open_webpage         | LOW       | No            | Fetches URL, extracts readable article text via Mozilla Readability. Max 5MB download limit. |
| create_note          | LOW       | No            | Inserts a personal note with optional topic tags into SQLite.                                |
| search_notes         | LOW       | No            | Performs keyword substring search across all saved personal notes.                           |
| update_note          | LOW       | No            | Updates the title, body, or tags of an existing personal note.                               |
| delete_note          | HIGH      | Yes           | Permanently deletes a note from SQLite. Requires user confirmation.                          |
| create_task          | LOW       | No            | Creates a task with priority (LOW, MED, HIGH) and optional due date.                         |
| list_tasks           | LOW       | No            | Lists tasks filtered by completion status (PENDING, COMPLETED).                              |
| update_task          | LOW       | No            | Modifies task description, priority, or due date.                                            |
| complete_task        | LOW       | No            | Marks a task as completed with an updated timestamp.                                         |
| delete_task          | HIGH      | Yes           | Permanently deletes a task from the database. Requires user confirmation.                    |
| create_reminder      | LOW       | No            | Schedules a notification reminder with optional recurrence.                                  |
| list_reminders       | LOW       | No            | Lists active or past reminder notifications.                                                 |
| complete_reminder    | LOW       | No            | Marks a reminder as completed.                                                               |
| dismiss_reminder     | LOW       | No            | Dismisses a firing reminder.                                                                 |
| remember_fact        | LOW       | No            | Stores an explicit cross-conversation personal preference or fact.                           |
| recall_facts         | LOW       | No            | Searches saved persistent memories.                                                          |
| forget_fact          | HIGH      | Yes           | Permanently erases a remembered fact. Requires user confirmation.                            |
| launch_application   | MEDIUM    | Yes           | Launches an approved desktop program (Notepad, Calculator, Paint). Strict allowlist.         |
| open_file            | MEDIUM    | Yes           | Opens an indexed document in its OS default viewer via os.startfile.                         |

|             |      |     |                                                                  |
|-------------|------|-----|------------------------------------------------------------------|
| lock_system | HIGH | Yes | Immediately locks the Windows session via Win32 LockWorkStation. |
|-------------|------|-----|------------------------------------------------------------------|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

## 8. Local RAG & Hybrid Retrieval Pipeline

JARVIS incorporates an end-to-end local Retrieval-Augmented Generation (RAG) pipeline designed to index and search personal documents without sending text to cloud embedding APIs.

### 1. Ingestion & Change Detection (`rag/ingestion.py`)

- Filesystem Crawl:** Recursively traverses configured folders while skipping hidden/system directories (e.g., `.git`, `node_modules`, `__pycache__`, `AppData`).
- Format Validation:** Validates file extensions against `DEFAULT_SUPPORTED_EXTENSIONS` (`.pdf`, `.docx`, `.pptx`, `.xlsx`, `.txt`, `.md`, `.csv`, `.json`, `.html`, `.py`, etc.).
- SHA-256 Change Detection:** Computes the SHA-256 hash of file contents and compares it against the stored hash in the `documents` table. Unmodified files are skipped completely, enabling instantaneous incremental reindexing.
- Format Parsing:** Dispatches the file to the appropriate extractor in `documents/registry.py`.

### 2. Token Chunking (`rag/chunking.py`)

Extracted document segments are partitioned into uniform chunks using a sliding window algorithm. Default settings: **650 tokens per chunk** with a **75-token overlap**. Chunks maintain references to source metadata, including file path, document ID, page numbers, and section headers.

### 3. Dense Vector Embeddings (`rag/embeddings.py`)

Embeddings are generated locally using the `sentence-transformers` library running the `all-MiniLM-L6-v2` model on CPU. It maps text chunks into a 384-dimensional dense vector space. Chunks are batched (default 32 chunks per batch) to optimize CPU cache locality.

### 4. Vector Storage (`rag/vector_store.py`)

Dense vectors are stored and queried using an in-memory **FAISS IndexFlatL2** index. The index maintains an associated metadata array mapping vector positions to SQLite chunk primary keys. The index is persisted to disk at `data/indexes/documents.index` using FAISS native serialization.

### 5. Sparse Keyword Storage (`rag/keyword_store.py`)

Concurrently, text chunks are indexed in SQLite using an **FTS5 virtual table** (`document_chunks_fts`). Keyword matching uses SQLite's built-in `bm25()` ranking function, providing exact token matching, acronym search, and code identifier lookup.

### 6. Hybrid Fusion & Cross-Encoder Reranking (`rag/hybrid_retriever.py`)

When a search query arrives, the hybrid retriever merges the results:

- Executes BM25 keyword query on SQLite FTS5 (retrieving top-N candidates).
- Executes L2 nearest neighbor vector search on FAISS (retrieving top-N candidates).
- Applies **Min-Max Score Normalization** to scale both BM25 and L2 distance scores to a uniform [0.0, 1.0] range.
- Computes combined hybrid score: `score = (0.5 * bm25_norm) + (0.5 * dense_norm)`.
- Passes the top 15 candidate chunks to a local cross-encoder model (`cross-encoder/ms-marco-MiniLM-L-6-v2` in `rag/reranker.py`) which computes deep query-passage attention scores, outputting the final 5 highest-precision chunks.

## 9. SQLite Storage Layer & Schema Migrations

All structured data is maintained in a single SQLite 3 database (`data/jarvis.db`). The storage layer in `storage/database.py` applies production-grade pragmas upon connection:

```
PRAGMA journal_mode = WAL;           -- Write-Ahead Logging for non-blocking reads/writes
PRAGMA synchronous = NORMAL;         -- Balances durability with disk write throughput
PRAGMA foreign_keys = ON;            -- Strict relational referential integrity
PRAGMA busy_timeout = 5000;          -- 5-second lock timeout prevents busy errors
```

### Database Schema: All 12 Tables

| Table Name          | Primary Key       | Key Columns & Relationships                                                                          | Purpose                                                       |
|---------------------|-------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| schema_migrations   | version (INTEGER) | applied_at (TIMESTAMP)                                                                               | Tracks applied database migration scripts.                    |
| conversations       | id (TEXT/UUID)    | title, created_at, updated_at                                                                        | Represents discrete conversational chat sessions.             |
| messages            | id (TEXT/UUID)    | conversation_id (FK), role (user/assistant/tool), content, citations (JSON), token_count, created_at | Stores chat history and tool response messages.               |
| indexed_folders     | id (INTEGER)      | path (UNIQUE), display_name, enabled, created_at, updated_at                                         | Folders configured by user for local RAG indexing.            |
| documents           | id (INTEGER)      | folder_id (FK), relative_path, file_hash, status (indexed/failed), error_message, updated_at         | Tracked files with SHA-256 change hashes.                     |
| document_chunks     | id (INTEGER)      | document_id (FK), chunk_index, text, page_number, section_name, token_count                          | Individual text chunks for search retrieval.                  |
| document_chunks_fts | rowid (INTEGER)   | text, tokenize='porter unicode61'                                                                    | SQLite FTS5 virtual table synchronized via database triggers. |
| notes               | id (INTEGER)      | title, content, tags (JSON), created_at, updated_at                                                  | Personal user notes.                                          |
| tasks               | id (INTEGER)      | description, status (PENDING/COMPLETED), priority (LOW/MED/HIGH), due_date                           | Personal user tasks and action items.                         |
| reminders           | id (INTEGER)      | message, remind_at, recurrence (daily/weekly), status (PENDING/FIRED/DISMISSED)                      | Scheduled notifications.                                      |
| memories            | id (INTEGER)      | fact, category, confidence, created_at                                                               | Long-term user preferences remembered across conversations.   |
| audit_log           | id (INTEGER)      | event_type, action, status (SUCCESS/ERROR), details (JSON), duration_ms, created_at                  | Security and operational activity audit ledger.               |
| llm_usage           | id (INTEGER)      | session_id, provider, model, prompt_tokens, completion_tokens, cost_usd, created_at                  | Detailed per-call LLM token and cost tracking.                |
| voice_usage         | id (INTEGER)      | session_id, service_type (STT/TTS), provider, model, units_consumed, cost_usd                        | Voice billing metrics (audio seconds / characters).           |

# 10. Web Search & Live Page Extraction

JARVIS enables the assistant to access live web information without relying on proprietary, privacy-invasive search APIs:

- **SearXNG Client** (`web/search.py`): Connects to a local or private SearXNG instance using its JSON search endpoint (GET `/search?format=json&q=...`). It aggregates results from Google, Bing, DuckDuckGo, and Wikipedia, stripping all tracking parameters.
- **SearXNG Process Manager** (`web/searxng_process.py`): Implements lazy lifecycle management for SearXNG. The process does not launch on application startup; instead, it is spawned automatically upon the first invocation of the `web_search` tool.
- **Cached Web Crawler** (`web/crawler.py`): Fetches remote pages using `httpx.AsyncClient` with a custom descriptive User-Agent. It enforces a strict **5 MB maximum payload cap** to prevent denial-of-service memory spikes and caches responses in an in-memory TTL cache (1-hour expiration) to eliminate redundant downloads.
- **Readability Extraction** (`web/extraction.py`): Strips navigation bars, headers, footers, advertisement iframes, and cookie banners using `readability-lxml` (the Python port of Mozilla's Firefox Reader Mode). The extracted article text is capped at 20,000 characters before delivery to the agent.

# 11. Voice Subsystem (Local & Cloud Hybrid)

JARVIS features a low-latency, push-to-talk voice pipeline supporting both cloud acceleration and 100% offline local inference:

## Audio Capture & Silence Gate (`voice/audio.py`)

- 1. **Push-to-Talk Capture:** When the user holds down the mic button, `AudioRecorder` initializes a 16,000 Hz, 16-bit mono input stream using `sounddevice` (PortAudio).
- 2. **RMS Energy Silence Gate:** Whisper speech recognition models have a known tendency to hallucinate phrases (e.g., "Thank you.", "Bye.", "Subtitles by...") when provided with pure silence or low ambient noise. JARVIS eliminates this by evaluating Root-Mean-Square (RMS) audio energy and duration. Any recording shorter than 0.5s or below the energy threshold is discarded before invoking STT backends.

## STT & TTS 3-Stage Fallback Chains

Both speech-to-text and text-to-speech operate on automatic provider failover chains:

| Stage                   | Speech-to-Text (STT)        | Text-to-Speech (TTS)           | Latency & Privacy Characteristics                  |
|-------------------------|-----------------------------|--------------------------------|----------------------------------------------------|
| Stage 1 (Cloud Primary) | Groq Whisper Large v3 Turbo | Groq PlayAI Orpheus            | Ultralow latency (~300ms), requires GROQ_API_KEY.  |
| Stage 2 (Cloud Backup)  | Deepgram Nova-3             | Deepgram Aura (Asteria)        | High accuracy, requires DEEPGRAM_API_KEY.          |
| Stage 3 (Local Offline) | faster-whisper (base, int8) | Piper TTS (lessac-medium ONNX) | 100% local, zero internet dependency, \$0.00 cost. |

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

## 12. Security, Secret Masking & Audit Logging

### Secret Scrubbing (`security/secrets.py`)

The system automatically gathers all secret patterns from the environment (e.g., keys starting with `gsk_`). Before any text is written to `jarvis.log`, saved to the `audit_log` table, or displayed in the UI, it passes through `scrub_secrets()`, which masks keys (e.g., `gsk_1234...abcd` → `gsk_...cd`).

### Non-Raising Audit Logging (`security/audit.py`)

Security auditing is non-intrusive. The `audit_action()` context manager wraps core operations, timing their execution and writing success/failure records to SQLite. If database auditing encounters an unexpected I/O error, it catches the exception and logs it to standard error without crashing the user's active session.

### Prompt Injection Defense

Retrieved content from local documents, web pages, and tool results is treated as untrusted data. The system prompt explicitly instructs the model to treat external content as reference data rather than executable instructions. Crucially, this defense is reinforced by hard application-level permission checks that cannot be bypassed via prompt injection.

# 13. PySide6 Desktop User Interface

The JARVIS UI is built with **PySide6 (Qt 6)**, providing a responsive, native Windows desktop experience:

- **Chat Window (`ui/chat_window.py`):** A sleek, frameless-style chat dialog featuring a custom header bar, presence dot, scrollable message history, auto-expanding input box, push-to-talk microphone button, and status indicator.
- **Message Bubbles & Markdown Rendering (`ui/messages.py`, `ui/markdown_render.py`):** User messages appear in distinct teal bubbles on the right; assistant responses render on the left with formatted Markdown tables, syntax-highlighted code snippets, bold text, lists, and hyperlinks.
- **Presence Dot Animation:** A visual status dot pulses in real time to reflect assistant states: Idle (gray), Thinking (pulsing cyan), Listening (pulsing green), Speaking (pulsing purple), or Error (red).
- **Interactive Citation Chips:** Tool results containing file references or web links render as interactive chips below assistant messages. Clicking a file chip opens the file in Windows; clicking a web chip opens the URL in the default browser.
- **Modal Confirmation Prompts:** When a HIGH-risk action is proposed, the UI displays an alert modal detailing the action, parameters, and risk level, providing explicit "Approve" and "Deny" choices.
- **Auxiliary Dialogs:** Includes a Settings dialog (folder management, startup toggle), History dialog (past conversation browser), and Usage & Costs dialog (session and all-time token/cost analytics).



# 14. Windows OS Integration

- **Global System Hotkey** (`ui/hotkey.py`): Registers Ctrl+Space via the low-level `keyboard` library, allowing users to summon JARVIS from any application.
- **Clean Startup Registration** (`app/windows_startup.py`): Rather than modifying registry Run keys or placing shortcuts in the Startup folder, JARVIS utilizes Windows Task Scheduler via `schtasks.exe` to create a clean, per-user logon task pointing to the lightweight supervisor process.
- **Application Allowlist** (`tools/windows.py`): Prevents arbitrary code execution by restricting the `launch_application` tool to a strict allowlist of standard Windows applications (Notepad, Calculator, Paint, Explorer, Task Manager).
- **Workstation Lock**: Integrates directly with Windows user session security via `ctypes.windll.user32.LockWorkStation()`.

# 15. Document Parsing Architecture

The document parsing subsystem (`documents/`) provides unified text extraction across diverse file formats via the `DocumentParser` protocol:

```
class DocumentParser(Protocol):
    def parse(self, path: Path) -> ExtractedDocument: ...
```

Each parser extracts structured text segmented by page or section:

- `documents/pdf.py`: Uses PyMuPDF (fitz) for page-by-page text and table extraction.
- `documents/docx.py`: Uses python-docx for paragraph and table content extraction.
- `documents/pptx.py`: Uses python-pptx for slide-by-slide shape and note text extraction.
- `documents/xlsx.py`: Uses openpyxl for spreadsheet rows converted to structured tab-delimited text.
- `documents/text.py`: Handles plain text, Markdown, CSV, JSON, source code files, and HTML (via BeautifulSoup tag stripping).

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|--|--|--|--|

# 16. Background Scheduler & Reminders

`scheduler/service.py` wraps APScheduler's `BackgroundScheduler` to deliver reliable desktop reminders:

- Polls the `reminders` table every 30 seconds on a background thread.
- Identifies pending reminders whose `remind_at` timestamp has passed.
- Marks the reminder as FIRED and emits a Qt signal to the main GUI thread.
- The GUI thread dispatches a native Windows toast notification (via `ui/notifications.py`) and appends a reminder notice directly into the chat stream.
- Supports recurring reminders (daily, weekly) by automatically calculating the next due timestamp.

# 17. End-to-End Execution Trace: A Complete Turn

To illustrate how all subsystems collaborate in real time, consider what happens when a user types: *"What are the latest updates on Claude Fable?"* and hits Enter:

1. **UI Capture:** `ChatWindow._on_send_clicked()` captures user text, appends a right-aligned teal message bubble to the chat log, clears the input box, sets presence dot to "Thinking", and spawns `ChatTurnWorker`.
2. **Worker Thread Launch:** `ChatTurnWorker.run()` executes in a background thread, invoking `agent.run_turn(conversation_id, text)`.
3. **Preprocess Node:** Trims text, validates inputs, initializes `AgentState`.
4. **Generate Node Initialization:**
  - Persists the user message to SQLite `messages` table.
  - Queries `messages` repository for the last 20 messages.
  - Queries `memories` repository for up to 30 relevant personal facts.
  - Constructs system prompt in `agent/prompts.py` with date, time, persona, and facts.
5. **Tool Loop Iteration 1 (LLM Reasoning):**
  - Calls Groq API (gpt-oss-120b) with conversation history + 17 tool JSON schemas.
  - Groq determines external knowledge is needed and emits a tool call: `web_search(query="Claude Fable updates")`.
  - Agent normalizes the tool call and checks permissions via `security/permissions.py` (Result: LOW risk, allowed).
  - Executes `web_search` tool → queries SearXNG → returns 8 search snippets.
  - Appends tool result message to the LLM message history.
6. **Tool Loop Iteration 2 (Deep Page Inspection):**
  - Calls Groq API with updated history containing search snippets.
  - Groq analyzes snippets and decides to inspect top sources via: `open_webpage(url="https://...")`.
  - Agent verifies permission (LOW risk, allowed).
  - Executes `open_webpage` tool → downloads HTML → extracts readable article text via Mozilla Readability → returns clean text.
  - Appends article content to message history.
7. **Tool Loop Iteration 3 (Final Synthesis):**
  - Calls Groq API with comprehensive article context.
  - Groq synthesizes the final answer formatted in rich Markdown (including update summary tables, bullet points, and release dates). Emits zero tool calls (`finish_reason: "stop"`).
8. **Audit Node & Persistence:**
  - Extracts web URL citations from tool results.
  - Saves assistant response + citations to `messages` table.
  - Records token consumption to `llm_usage` table.
  - Records complete turn duration to `audit_log` table.
9. **UI Presentation:**
  - `ChatTurnWorker` emits `succeeded` Qt signal with final state dict.
  - Main GUI thread receives signal in `_on_turn_succeeded()`.
  - Renders assistant response as HTML via `render_markdown()` in `ui/markdown_render.py`.
  - Appends clickable citation chips below the bubble.
  - Resets presence dot to "Idle" and re-enables text input.

# 18. Complete File-by-File Technical Reference

This chapter presents an exhaustive, file-by-file analysis of every source file in the JARVIS repository. For each file, the reference details its exact filesystem location, its primary architectural responsibility, and a deep technical code walkthrough including classes, functions, algorithms, and error handling mechanisms.

## 18.1 Application Lifecycle & Supervisor (`app/`)

### `app/main.py`

Path: `app/main.py` • Size: ~522 bytes • Lines: 26

**What it does:** Serves as the primary entry point for the interactive JARVIS application. Orchestrates the startup sequence, initializes the Qt GUI, and ensures graceful teardown upon application exit.

#### Code Explanation:

- `main()`: The main execution function invoked when running `python -m app.main`.
- Calls `bootstrap()` from `app.bootstrap` to initialize core configuration, logging, database connections, and migrations.
- Invokes `run_health_checks(ctx.settings)` from `app.lifecycle` to inspect dependencies and determine if any component is degraded.
- Invokes `ui.chat_window.run(health_warnings=warnings)`, which creates the PySide6 `QApplication`, displays the chat window, and enters the Qt event loop.
- Wraps the GUI lifecycle in a `try...finally` block, ensuring that `shutdown()` is called when the Qt application exits, terminating background worker processes and closing database handles.

### `app/bootstrap.py`

Path: `app/bootstrap.py` • Size: ~2,926 bytes • Lines: 90

**What it does:** Encapsulates application pre-flight configuration and startup wiring into a single repeatable function. Prepares all data directories, database schemas, and shared singleton managers.

#### Code Explanation:

- `BootstrapContext`: A typed dataclass holding references to `settings`, `db_connection`, and `searxng_manager`.
- `_configure_logging(settings)`: Sets up dual log destinations. Attaches a `StreamHandler` for console standard output and a `RotatingFileHandler` directed to `data/logs/jarvis.log` with a 5MB size limit and 3 backups. Applies secret scrubbing formatters.
- `bootstrap()` -> `BootstrapContext`: The main initialization routine:
  1. Retrieves singleton `Settings` via `get_settings()`.
  2. Creates required directories (`data_dir`, `logs_dir`, `indexes_dir`, `cache_dir`, `voices_dir`).
  3. Calls `storage.database.connect()` to open SQLite in WAL mode.
  4. Runs `run_migrations(conn)` to apply any outstanding DDL changes.
  5. Initializes the `SearXNGProcessManager` singleton without immediately spawning the process.
  6. Logs an audit event (`STARTUP`) recording the timestamp, host, and version.

### `app/lifecycle.py`

Path: `app/lifecycle.py` • Size: ~7,202 bytes • Lines: 185

**What it does:** Implements comprehensive pre-flight health diagnostic checks for all 8 subsystems and orchestrates orderly application shutdown.

#### Code Explanation:

- `ComponentState`: Enum indicating subsystem health (`HEALTHY`, `DEGRADED`, `FAILED`).
- `HealthCheckResult`: Dataclass containing `component name`, `state`, and human-readable detail.

- **Individual check functions:**
  - `_check_config()`: Validates that critical configuration fields are parsed properly.
  - `_check_database()`: Executes a test query (`SELECT 1`) and inspects table counts.
  - `_check_vector_index()`: Verifies the existence and readability of the FAISS index file.
  - `_check_llm_provider()`: Verifies that at least one LLM API key (Groq or Ollama) is configured.
  - `_check_web_search()`: Performs a lightweight HTTP ping against SearXNG with a 2-second timeout.
  - `_check_voice()`: Tests `sounddevice` audio input devices and Piper voice files.
  - `_check_scheduler()`: Verifies APScheduler service readiness.
  - `_check_ui()`: Confirms PySide6 display server bindings.
- `run_health_checks(settings) -> list[HealthCheckResult]`: Runs all checks concurrently, catching exceptions so diagnostics never block app launch.
- `summarize_health_warnings(results) -> str | None`: Formats degraded/failed results into a concise string for display in the UI banner.
- `shutdown()`: Stops the SearXNG process manager, shuts down background worker pools, logs a `SHUTDOWN` audit event, and flushes log buffers.

### app/supervisor.py

Path: `app/supervisor.py` • Size: ~5,437 bytes • Lines: 150

**What it does:** The lightweight, always-on Windows daemon (~3MB RAM) that registers the global shortcut (Ctrl+Space) and manages single-instance activation of the full JARVIS chat window.

#### Code Explanation:

- Contains zero heavy imports — only `ctypes`, `subprocess`, `keyboard`, and basic PySide6 GUI widgets for the tray icon.
- `bring_window_to_front() -> bool`: Uses `Win32 user32.FindWindowW(None, "JARVIS")` to locate an existing chat window. If found, calls `user32.ShowWindow(hwnd, 9)` (`SW_RESTORE`) and `user32.SetForegroundWindow(hwnd)` to focus the window.
- `launch_or_focus()`: Callback triggered when Ctrl+Space is pressed.
  - First attempts `bring_window_to_front()`.
  - If no window exists, checks an 8-second debounce timer (`_last_launch_time`) to prevent accidental double-launches.
  - Spawns the main app via `subprocess.Popen(["cmd.exe", "/c", "start_jarvis.bat"])` with `CREATE_NO_WINDOW` flag.
- `SupervisorTray`: Creates a minimal tray icon with "Open JARVIS" and "Exit Supervisor" menu options.

### app/qt\_sigint.py

Path: `app/qt_sigint.py` • Size: ~1,357 bytes • Lines: 34

**What it does:** Solves the classic problem where pressing Ctrl+C in a terminal does not terminate a Qt application on Windows or Linux.

#### Code Explanation:

- Because the Qt C++ event loop does not periodically yield control to the Python interpreter, standard Python signal handlers for `SIGINT` are blocked indefinitely.
- `setup_sigint_handler(app: QApplication)`:
  1. Installs a Python `signal.signal(signal.SIGINT, ...)` handler that calls `app.quit()`.
  2. Instantiates a `QTimer` that fires every 200 milliseconds with an empty callback (`lambda: None`). This forces Qt to periodically yield execution to Python bytecode, enabling Python to process pending OS signals promptly.

### app/windows\_startup.py

Path: `app/windows_startup.py` • Size: ~3,168 bytes • Lines: 85

**What it does:** Manages automatic startup with Windows using Task Scheduler (`schtasks.exe`) rather than fragile registry Run keys or Startup shortcuts.

#### Code Explanation:

- `TASK_NAME = "JARVIS_Supervisor"`: The identifier used in Task Scheduler.
- `is_startup_enabled() -> bool`: Executes `schtasks /query /tn JARVIS_Supervisor` via `subprocess.run()`. Returns `True` if the return code is 0.
- `enable_startup() -> bool`: Registers a logon task for the current user: `schtasks /create /tn JARVIS_Supervisor /tr "path\to\start_jarvis_supervisor.bat" /sc onlogon /rl limited /f`. Targets the lightweight supervisor, not the heavy main app.
- `disable_startup() -> bool`: Unregisters the task via `schtasks /delete /tn JARVIS_Supervisor /f`.

#### app/\_\_\_init\_\_\_py

Path: `app/___init___py` • Size: ~0 bytes • Lines: 0

**What it does:** Python package marker file designating the `app` directory as an importable module.

## 18.2 Agent Orchestration (`agent/`)

#### agent/graph.py

Path: `agent/graph.py` • Size: ~17,767 bytes • Lines: 447

**What it does:** Implements the cognitive core of JARVIS using a `LangGraph StateGraph`. Coordinates the iterative tool execution loop, provider pinning, citation extraction, and risk confirmation flows.

#### Code Explanation:

- `Agent` class: The central controller instantiated with `llm_manager`, `tool_registry`, and `db_connection`.
- `_build_graph()`: Assembles the `LangGraph` state machine: `workflow.add_node("preprocess", self._preprocess_node) -> workflow.add_node("generate", self._generate_node) -> workflow.add_node("audit", self._audit_node)` with linear edges.
- `_run_tool_loop(state, messages)`: The heart of the reasoning process:
  1. Loops up to `DEFAULT_AGENT_MAX_ITERATIONS` (8 cycles).
  2. Calls `self.llm_manager.generate()` with current messages and tool schemas.
  3. Pins the responding provider to prevent incompatible tool call serialization in subsequent iterations.
  4. If no tool calls are emitted, sets `state["response"] = content` and breaks the loop.
  5. If tool calls are returned, normalizes arguments via `_normalize_tool_calls()`.
  6. Checks permissions via `security.permissions.check_permission(tool)`.
  7. If a tool is HIGH-risk, sets `state["requires_confirmation"] = True` and `state["confirmation_request"] = {...}`, then immediately aborts the loop.
  8. Executes allowed tools via `asyncio.wait_for(tool.handler(**args), timeout)`, packages output as a tool-result message, and appends to the conversation context.
- `confirm_and_execute(turn_state, approved)`: Handles user response to a HIGH-risk confirmation prompt. If approved, executes the exact cached tool call directly without re-invoking the LLM, persists the result, and returns an updated state.
- `_extract_citations(tool_results)`: Inspects tool outputs for file paths and web URLs, deduplicating them into structured citation objects.

#### agent/prompts.py

Path: `agent/prompts.py` • Size: ~4,705 bytes • Lines: 90

**What it does:** Assembles the dynamic system prompt that governs assistant persona, behavioral constraints, safety boundaries, and contextual awareness.

#### Code Explanation:

- `BASE_SYSTEM_PROMPT`: Defines JARVIS's identity as an efficient, highly capable personal assistant. Explicitly lays out prompt injection defense rules, instructing the model to treat retrieved documents and web pages as untrusted data rather than instructions.
- `build_system_prompt(settings, db_conn=None) -> str`:
  1. Injects user preferences: `JARVIS_USER_NAME` and `JARVIS_TIMEZONE`.
  2. Injects dynamic runtime context: current localized date, time, and day of week.
  3. Queries `storage.repositories.memories.list_memories()` to retrieve up to 30 active facts and appends them as an itemized memory block.

### agent/state.py

Path: `agent/state.py` • Size: ~975 bytes • Lines: 39

**What it does:** Defines the strongly-typed state schema passed between LangGraph nodes during a conversational turn.

#### Code Explanation:

- `AgentState(TypedDict)`: Contains fields:
  - `user_message: str` — The raw input text from the user.
  - `conversation_id: str` — UUID of the active conversation.
  - `intent: str | None` — Optional classified intent tag.
  - `plan: list[str] | None` — Multi-step task plan if formulated.
  - `tool_calls: list[dict]` — Tool calls emitted by the LLM during the turn.
  - `tool_results: list[dict]` — Execution outputs returned by tools.
  - `retrieved_context: list[str]` — RAG context chunks retrieved for the turn.
  - `citations: list[dict]` — Structured citation chips (file paths, URLs).
  - `response: str | None` — Final text output produced for the user.
  - `requires_confirmation: bool` — Flag indicating a HIGH-risk action is pending.
  - `confirmation_request: dict | None` — Details of the proposed risky action.
  - `error: str | None` — Error message if an unhandled failure occurred.
- `new_state(conversation_id, user_message) -> AgentState`: Factory function returning an initialized state dictionary with default empty values.

### agent/\_\_init\_\_.py

Path: `agent/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `agent` module.

## 18.3 Configuration Subsystem (`config/`)

### config/defaults.py

Path: `config/defaults.py` • Size: ~7,016 bytes • Lines: 151

**What it does:** Central repository for all application constants, thresholds, timeouts, model identifiers, and retrieval weights. Enforces the strict "no magic numbers" rule.

#### Code Explanation:

- Declares over 60 fully documented constant variables grouped by subsystem:
  - **LLM:** `DEFAULT_LLM_TIMEOUT_SECONDS = 60.0`, `DEFAULT_LLM_RATE_LIMIT_MAX_WAIT_SECONDS = 8.0`, `DEFAULT_PRIMARY_MODEL = "openai/gpt-oss-120b"`, `DEFAULT_FALLBACK_MODEL = "gpt-oss:120b"`.
  - **Agent:** `DEFAULT_AGENT_MAX_ITERATIONS = 8`, `DEFAULT_MAX_HISTORY_MESSAGES = 20`, `DEFAULT_MAX_MEMORIES_INJECTED = 30`.
  - **RAG:** `DEFAULT_CHUNK_SIZE_TOKENS = 650`, `DEFAULT_CHUNK_OVERLAP_TOKENS = 75`, `DEFAULT_EMBEDDING_MODEL = "all-MiniLM-L6-v2"`, `DEFAULT_RERANKER_MODEL = "cross-encoder/ms-`



```
marco-MiniLM-L-6-v2", DEFAULT_HYBRID_BM25_WEIGHT = 0.5, DEFAULT_HYBRID_DENSE_WEIGHT = 0.5.
```

- **Web:** DEFAULT\_SEARXNG\_URL = "http://localhost:8080", DEFAULT\_CRAWLER\_MAX\_BYTES = 5\_242\_880 (5MB), DEFAULT\_EXTRACTION\_MAX\_CHARS = 20000.
- **Voice:** DEFAULT\_AUDIO\_SAMPLE\_RATE = 16000, DEFAULT\_SILENCE\_RMS\_THRESHOLD = 0.01, DEFAULT\_SILENCE\_MIN\_SECONDS = 0.5.
- **Windows:** DEFAULT\_APP\_ALLOWLIST = ["notepad.exe", "calc.exe", "mspaint.exe", "explorer.exe"].

### config/settings.py

Path: config/settings.py • Size: ~7,941 bytes • Lines: 181

**What it does:** Defines the strongly-typed `Settings` class using Pydantic Settings. Reads and validates `.env` configurations and dynamically computes derived directory paths.

#### Code Explanation:

- `Settings(BaseSettings):`
  - Reads from `.env` with UTF-8 encoding.
  - Defines typed fields for API keys (`groq_api_key`, `ollama_cloud_api_key`, `deepgram_api_key`).
  - Defines feature flags (`jarvis_enable_voice`, `jarvis_enable_reranker`).
  - Defines provider sequences (`jarvis_stt_providers = "groq, deepgram, local"`).
  - Derived filesystem properties: `data_dir`, `db_path`, `logs_dir`, `indexes_dir`, `cache_dir`, `voices_dir`.
- `get_settings() -> Settings:` Decorated with `@lru_cache()`, returning a shared validated singleton across the entire process lifetime.

### config/\_\_init\_\_.py

Path: config/\_\_init\_\_.py • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `config` module.

## 18.4 LLM Provider Subsystem (llm/)

### llm/base.py

Path: llm/base.py • Size: ~4,453 bytes • Lines: 142

**What it does:** Establishes the foundational provider abstractions, unified response data structures, and the structured provider exception hierarchy.

#### Code Explanation:

- `TokenUsage(BaseModel):` Tracks `prompt_tokens`, `completion_tokens`, `total_tokens`, and `estimated_cost_usd`.
- `LLMResponse(BaseModel):` Standardized model response containing `content`, `provider`, `model`, `finish_reason`, `tool_calls`, `raw`, and `usage`.
- `LLMProvider(Protocol):` Runtime-checkable Python protocol defining `async def generate(messages, tools=None) -> LLMResponse`.
- **Exception Hierarchy:** `ProviderError` → `ProviderConnectionError`, `ProviderTimeoutError`, `ProviderRateLimitError` (carries `retry_after: float`), `ProviderAuthError`, `ProviderResponseError`.
- `AllProvidersFailedError:` Raised when every configured provider fails, bundling each provider's error for debugging.
- `parse_retry_after_header(value: str) -> float | None:` Parses standard numeric seconds or HTTP dates from rate-limit response headers.

### llm/groq\_provider.py

Path: llm/groq\_provider.py • Size: ~8,409 bytes • Lines: 227

**What it does:** Implements the primary LLM provider connecting to Groq's high-speed inference API. Features exponential backoff and custom compound duration rate-limit parsing.

**Code Explanation:**

- `GroqProvider (LLMProvider)`: Initialized with `api_key`, `model`, and `timeout`. Uses an asynchronous `httpx.AsyncClient`.
- `_parse_groq_duration(s: str) -> float`: Uses regular expressions to parse Groq's proprietary rate-limit strings (e.g., "547ms" → 0.547s, "1.065s" → 1.065s, "8m38.4s" → 518.4s).
- `generate(messages, tools=None)`:
  1. Serializes messages and OpenAI-format tool schemas.
  2. Executes POST `https://api.groq.com/openai/v1/chat/completions`.
  3. Catches HTTP 429: extracts reset duration. If wait ≤ 8s, sleeps via `asyncio.sleep()` and retries. If wait > 8s, raises `ProviderRateLimitError` immediately to trigger failover.
  4. Calculates token usage and estimated cost using `llm.pricing`.
  5. Returns populated `LLMResponse`.

**llm/ollama\_cloud\_provider.py**

Path: llm/ollama\_cloud\_provider.py • Size: ~8,873 bytes • Lines: 229

**What it does:** Implements the secondary fallback provider connecting to Ollama Cloud's hosted endpoint. Handles schema translations between OpenAI format and Ollama's native JSON dictionary format.

**Code Explanation:**

- `OllamaCloudProvider (LLMProvider)`: Connects to Ollama Cloud via POST `/api/chat`.
- `_translate_messages(messages)`: Converts OpenAI-style message structures into Ollama format. Specifically converts stringified function arguments (e.g., `"{\\"query\\": \\"abc\\"}"`) into native Python dictionaries required by Ollama.
- `_translate_tools(tools)`: Formats tool schemas to match Ollama's expected function calling format.
- `generate(messages, tools=None)`: Dispatches translated request, handles connection and timeout errors, converts Ollama response format into standardized `LLMResponse`, and computes token metrics.

**llm/manager.py**

Path: llm/manager.py • Size: ~2,368 bytes • Lines: 64

**What it does:** Coordinates the provider chain, orchestrating seamless failover when the primary provider encounters rate limits, timeouts, or authentication errors.

**Code Explanation:**

- `LLMManager`: Holds an ordered list of providers: `list[LLMProvider]`.
- `from_settings(settings) -> LLMManager`: Factory constructing the default fallback sequence: `[GroqProvider, OllamaCloudProvider]` based on configured API keys.
- `generate(messages, tools=None, pinned_provider=None) -> LLMResponse`:
  1. If `pinned_provider` is specified, prioritizes that provider.
  2. Iterates through available providers in order.
  3. On `ProviderError`: logs a warning detailing the failure and immediately advances to the next provider.
  4. If all providers fail, raises `AllProvidersFailedError` bundling all underlying exceptions.

**llm/pricing.py**

Path: llm/pricing.py • Size: ~1,680 bytes • Lines: 40

**What it does:** Maintains per-model pricing rate cards and computes USD costs for prompt and completion token consumption.

**Code Explanation:**

- `PRICING_TABLE`: Dictionary mapping `(provider, model)` tuples to input/output rates per million tokens. For Groq `openai/gpt-oss-120b`: \$0.15 / 1M prompt tokens, \$0.60 / 1M completion tokens.
- `calculate_cost(provider, model, prompt_tokens, completion_tokens) -> float | None`: Returns exact

computed cost in USD. If the provider/model does not charge per token (such as subscription-based Ollama Cloud), returns `None` rather than a misleading zero.

### llm/\_\_init\_\_.py

Path: llm/\_\_init\_\_.py • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the llm module.

## 18.5 Storage & Database Layer (storage/)

### storage/database.py

Path: storage/database.py • Size: ~1,319 bytes • Lines: 46

**What it does:** Implements the core SQLite connection factory, configuring database pragmas for concurrent read/write performance and relational integrity.

#### Code Explanation:

- `connect(db_path: Path) -> sqlite3.Connection`: Opens a SQLite connection to `data/jarvis.db` with `check_same_thread=False`. Sets `row_factory = sqlite3.Row` so queries return dict-like rows with named column access.
- Applies essential pragmas: WAL mode (enables concurrent readers while writing), `synchronous = NORMAL`, and `foreign_keys = ON`.
- `get_connection()`: Context manager yielding an active connection, auto-committing on normal exit and auto-rolling back if an uncaught exception is raised.

### storage/migrations.py

Path: storage/migrations.py • Size: ~10,625 bytes • Lines: 283

**What it does:** Manages the forward-only, idempotent database schema migrations, ensuring the SQLite database structure stays synchronized with application models.

#### Code Explanation:

- Creates and maintains the `schema_migrations` version ledger table.
- Defines sequential migration scripts:
  - **Migration 1 (Initial Schema)**: Creates 10 core tables (`conversations`, `messages`, `indexed_folders`, `documents`, `document_chunks`, `notes`, `tasks`, `reminders`, `memories`, `audit_log`) with foreign keys and indexes.
  - **Migration 2 (FTS5 Full-Text Search)**: Creates the `document_chunks_fts` virtual table with Porter stemmer and unicode61 tokenizer. Establishes SQLite triggers (`after_insert`, `after_delete`, `after_update`) to keep FTS5 synchronized automatically.
  - **Migration 3 (LLM Usage & Billing)**: Creates the `llm_usage` table to track per-call token counts, session IDs, and costs.
  - **Migration 4 (Voice Usage & Billing)**: Creates the `voice_usage` table to track audio seconds and TTS character metrics.
- `run_migrations(conn)`: Queries applied versions, executes pending migrations within atomic transactions, and records completion timestamps.

### storage/models.py

Path: storage/models.py • Size: ~2,540 bytes • Lines: 126

**What it does:** Provides lightweight, typed Python dataclasses representing rows across all SQLite tables without the overhead of a heavy ORM.

#### Code Explanation:

- Defines dataclasses matching database schemas: `Conversation`, `Message`, `IndexedFolder`, `Document`, `DocumentChunk`, `Note`, `Task`, `Reminder`, `Memory`, `AuditEntry`, `LLMUsageRecord`, and `VoiceUsageRecord`.

- Includes factory classmethods (e.g., `from_row(row: sqlite3.Row)`) to deserialize database rows into strongly-typed objects with JSON field decoding.

### storage/\_\_init\_\_.py

Path: `storage/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `storage` module.

## 18.6 Storage Repositories (`storage/repositories/`)

### storage/repositories/conversations.py

Path: `storage/repositories/conversations.py` • Size: ~4,210 bytes • Lines: 135

**What it does:** Data access layer for conversations and message history, providing persistence, retrieval, and conversion to LLM-ready message payloads.

#### Code Explanation:

- `create_conversation(conn, title="New Conversation") -> str`: Generates a UUID, creates a row in `conversations`, and returns the conversation ID.
- `get_conversation(conn, id) -> dict | None`: Fetches conversation metadata by primary key.
- `list_conversations(conn, limit=50) -> list[dict]`: Retrieves recent conversations ordered by `updated_at` DESC.
- `add_message(conn, conversation_id, role, content, citations=None, token_count=0) -> str`: Appends a message, serializes citations to JSON, and updates the conversation's `updated_at` timestamp.
- `get_messages(conn, conversation_id, limit=20) -> list[dict]`: Retrieves recent messages in chronological order.
- `to_llm_messages(messages) -> list[dict]`: Formats database message records into OpenAI-compatible `{"role": ..., "content": ...}` dictionaries for LLM input.

### storage/repositories/documents.py

Path: `storage/repositories/documents.py` • Size: ~4,820 bytes • Lines: 148

**What it does:** Manages persistence for tracked files and their indexed text chunks.

#### Code Explanation:

- `upsert_document(conn, folder_id, relative_path, file_hash, status="indexed") -> int`: Inserts or updates document records based on unique `(folder_id, relative_path)`.
- `get_document_by_path(conn, folder_id, relative_path) -> dict | None`: Retrieves file tracking details.
- `delete_document_chunks(conn, document_id)`: Deletes all chunks associated with a document (triggers automatic clean up FTS5 entries).
- `insert_chunks(conn, document_id, chunks)`: Batch inserts chunks with page numbers, section headers, and token counts.
- `get_chunk_by_id(conn, chunk_id) -> dict | None`: Fetches chunk text and source file metadata for search result assembly.

### storage/repositories/folders.py

Path: `storage/repositories/folders.py` • Size: ~2,840 bytes • Lines: 92

**What it does:** Manages the list of user-approved indexed folders and validates file access security boundaries.

#### Code Explanation:

- `add_folder(conn, path, display_name=None) -> int`: Registers a directory path for indexing.
- `remove_folder(conn, folder_id)`: Unregisters a directory and cascades deletion across its tracked documents and chunks.
- `list_folders(conn, enabled_only=True) -> list[dict]`: Returns active directories.

- `is_path_within_indexed_folders(conn, target_path) -> bool`: Resolves absolute canonical paths and verifies whether `target_path` resides strictly inside an approved folder, enforcing the file sandbox.

### storage/repositories/memories.py

Path: `storage/repositories/memories.py` • Size: ~3,120 bytes • Lines: 101

**What it does:** Provides persistence for long-term personal facts remembered across independent conversation sessions.

#### Code Explanation:

- `create_memory(conn, fact: str, category="user_preference") -> int`: Stores a personal fact.
- `list_memories(conn, limit=30) -> list[dict]`: Returns active remembered facts for system prompt injection.
- `search_memories(conn, query: str) -> list[dict]`: Performs substring search across memories.
- `delete_memory(conn, memory_id: int) -> bool`: Permanently deletes a memory record.

### storage/repositories/notes.py

Path: `storage/repositories/notes.py` • Size: ~3,650 bytes • Lines: 114

**What it does:** CRUD operations for personal user notes with tag classification.

#### Code Explanation:

- `create_note(conn, title, content, tags=None) -> int`: Creates a note with JSON-serialized tags.
- `search_notes(conn, query) -> list[dict]`: Searches title and body content for matching keywords.
- `update_note(conn, note_id, title=None, content=None, tags=None) -> bool`: Modifies an existing note.
- `delete_note(conn, note_id) -> bool`: Permanently removes a note.

### storage/repositories/tasks.py

Path: `storage/repositories/tasks.py` • Size: ~3,890 bytes • Lines: 122

**What it does:** CRUD operations for task tracking, priorities, and completion states.

#### Code Explanation:

- `create_task(conn, description, priority="MED", due_date=None) -> int`: Creates a task with priority validation (LOW, MED, HIGH).
- `list_tasks(conn, status="PENDING") -> list[dict]`: Lists tasks filtered by status.
- `complete_task(conn, task_id) -> bool`: Sets task status to `COMPLETED` and records completion time.
- `update_task(conn, task_id, description=None, priority=None, due_date=None) -> bool`: Modifies task properties.
- `delete_task(conn, task_id) -> bool`: Deletes a task record.

### storage/repositories/reminders.py

Path: `storage/repositories/reminders.py` • Size: ~3,950 bytes • Lines: 128

**What it does:** Manages scheduled reminder notifications and recurring alarms.

#### Code Explanation:

- `create_reminder(conn, message, remind_at, recurrence=None) -> int`: Schedules a reminder for a specific timestamp.
- `get_due_reminders(conn, current_time) -> list[dict]`: Queries reminders where `status = 'PENDING'` and `remind_at <= current_time`.
- `mark_fired(conn, reminder_id)`: Updates reminder state to `FIRE`D. For recurring reminders, calculates the next timestamp and re-arms.
- `dismiss_reminder(conn, reminder_id) -> bool`: Dismisses an active notification.

### storage/repositories/usage.py

Path: `storage/repositories/usage.py` • Size: ~3,200 bytes • Lines: 98

**What it does:** Records token metrics and computes aggregate LLM expenditures for the active session and all-time history.

#### Code Explanation:

- `record_usage(conn, session_id, provider, model, prompt_tokens, completion_tokens, cost_usd)`: Logs an individual inference call.
- `get_session_usage(conn, session_id) -> dict`: Computes total prompt tokens, completion tokens, and dollar cost for a specific session.
- `get_all_usage(conn) -> dict`: Computes lifetime application token and cost totals broken down by provider.

#### storage/repositories/voice\_usage.py

Path: `storage/repositories/voice_usage.py` • Size: ~3,150 bytes • Lines: 96

**What it does:** Records voice processing billing metrics (STT audio seconds and TTS character counts).

#### Code Explanation:

- `record_stt_usage(conn, session_id, provider, model, audio_seconds, cost_usd)`: Logs speech-to-text usage.
- `record_tts_usage(conn, session_id, provider, model, character_count, cost_usd)`: Logs text-to-speech synthesis usage.
- `get_voice_usage_summary(conn) -> dict`: Returns aggregate audio seconds transcribed and characters synthesized with cost breakdowns.

#### storage/repositories/\_\_init\_\_.py

Path: `storage/repositories/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for `storage.repositories`.

## 18.7 Tool Subsystem (`tools/`)

#### tools/registry.py

Path: `tools/registry.py` • Size: ~2,723 bytes • Lines: 84

**What it does:** Provides the core tool registration mechanism, metadata definitions, and automatic generation of OpenAI-compatible function calling schemas.

#### Code Explanation:

- `RiskLevel(Enum)`: Defines `LOW`, `MEDIUM`, and `HIGH` risk tiers.
- `ToolMetadata(dataclass)`: Holds `name`, `description`, `risk_level`, `requires_confirmation: bool`, and `timeout_seconds: float`.
- `Tool(dataclass)`: Couples metadata, a JSON Schema `input_schema`, and an `async` execution handler.
- `ToolRegistry`:
  - `register(tool: Tool)`: Adds a tool to the internal dictionary.
  - `get(name: str) -> Tool | None`: Retrieves a tool by name.
  - `llm_schemas() -> list[dict]`: Converts registered tools into standard OpenAI function calling schema specifications (`{ "type": "function", "function": {...} }`).

#### tools/file\_search.py

Path: `tools/file_search.py` • Size: ~2,450 bytes • Lines: 75

**What it does:** Implements the `search_files` tool handler, connecting the agent to the local hybrid RAG retriever.

#### Code Explanation:

- Accepts `query: str` and optional `top_k: int = 5`.

- Invokes `HybridRetriever.retrieve(query, top_k)` to execute combined BM25 and dense vector search.
- Formats the highest-ranking chunks into structured text snippets with document filenames, page numbers, section titles, and relevance scores.

### tools/file\_reader.py

Path: tools/file\_reader.py • Size: ~2,890 bytes • Lines: 88

**What it does:** Implements the `read_file` tool handler, allowing the agent to inspect specific local files within the approved security boundary.

#### Code Explanation:

- Accepts `file_path: str`, `start_line: int = 1`, and `max_lines: int = 200`.
- Validates that `file_path` resides strictly inside an indexed folder via `is_path_within_indexed_folders()`. If outside, raises a security violation.
- Reads file content using UTF-8 (with fallback to Latin-1), slices requested line ranges, and returns text with line numbers.

### tools/file\_listing.py

Path: tools/file\_listing.py • Size: ~2,640 bytes • Lines: 82

**What it does:** Implements `list_folder` and `list_indexed_folders` tools for inspecting directory contents.

#### Code Explanation:

- `list_indexed_folders`: Returns all currently enabled folder paths that JARVIS is authorized to access.
- `list_folder(folder_path, max_depth=1)`: Checks security boundary, walks the directory, and returns a formatted tree of files and subfolders with byte sizes.

### tools/file\_writer.py

Path: tools/file\_writer.py • Size: ~2,950 bytes • Lines: 92

**What it does:** Implements the `write_file` tool for creating or modifying text files on disk. Classified as HIGH risk, requiring explicit user approval.

#### Code Explanation:

- Registered with `risk_level = RiskLevel.HIGH` and `requires_confirmation = True`.
- Accepts `file_path: str`, `content: str`, and `overwrite: bool = False`.
- Verifies that the target path is inside an indexed folder.
- Writes file atomically using a temporary file and atomic rename, preserving UTF-8 encoding.

### tools/web\_search.py

Path: tools/web\_search.py • Size: ~2,580 bytes • Lines: 78

**What it does:** Implements the `web_search` tool, enabling live web queries via local SearXNG.

#### Code Explanation:

- Accepts `query: str` and `max_results: int = 8`.
- Ensures SearXNG is running via `SearXNGProcessManager.ensure_running()`.
- Calls `SearXNGClient.search(query, max_results)` to execute the query.
- Returns formatted search snippets containing titles, URLs, and text excerpts.

### tools/web\_reader.py

Path: tools/web\_reader.py • Size: ~2,710 bytes • Lines: 85

**What it does:** Implements the `open_webpage` tool for deep article inspection.

#### Code Explanation:

- Accepts `url: str`.



- Fetches HTML via `web.crawler.fetch_html(url)` (using in-memory TTL cache and 5MB size limit).
- Extracts clean article text via `web.extraction.extract_readable_text()`.
- Returns title, domain, character count, and article text capped at 20,000 characters.

### tools/notes.py

Path: tools/notes.py • Size: ~3,820 bytes • Lines: 118

**What it does:** Implements tool handlers for note management: `create_note`, `search_notes`, `update_note`, and `delete_note`.

#### Code Explanation:

- `create_note`, `search_notes`, `update_note` are classified as LOW risk.
- `delete_note` is classified as HIGH risk (`requires_confirmation=True`).
- Handlers route parameters directly to the `storage.repositories.notes` module.

### tools/tasks.py

Path: tools/tasks.py • Size: ~3,950 bytes • Lines: 124

**What it does:** Implements tool handlers for tasks: `create_task`, `list_tasks`, `update_task`, `complete_task`, and `delete_task`.

#### Code Explanation:

- Task creation and completion are LOW risk; `delete_task` is HIGH risk.
- Handlers validate task priority (LOW, MED, HIGH) and date strings before database storage.

### tools/reminders.py

Path: tools/reminders.py • Size: ~3,740 bytes • Lines: 115

**What it does:** Implements tool handlers for reminder scheduling: `create_reminder`, `list_reminders`, `complete_reminder`, and `dismiss_reminder`.

#### Code Explanation:

- All reminder management tools are LOW risk.
- `create_reminder` resolves relative time expressions ("in 15 minutes", "tomorrow at 9 AM") into localized ISO-8601 timestamps using `JARVIS_TIMEZONE`.

### tools/memory.py

Path: tools/memory.py • Size: ~4,748 bytes • Lines: 139

**What it does:** Implements persistent cross-conversation memory tools: `remember_fact`, `recall_facts`, and `forget_fact`.

#### Code Explanation:

- `remember_fact(fact: str)`: Low risk; called only when the user explicitly requests remembering a preference.
- `recall_facts(query: str)`: Low risk; searches remembered facts.
- `forget_fact(fact_id: int)`: HIGH risk (`requires_confirmation=True`); permanently erases a stored fact.

### tools/windows.py

Path: tools/windows.py • Size: ~5,090 bytes • Lines: 148

**What it does:** Implements safe Windows desktop integration tools: `launch_application`, `open_file`, and `lock_system`.

#### Code Explanation:

- `launch_application(app_name: str)`: Classified as MEDIUM risk (requires user confirmation). Checks `DEFAULT_APP_ALLOWLIST` (Notepad, Calculator, Paint, Explorer, Task Manager). Rejects any executable not on the allowlist to prevent arbitrary code execution. Spawns via `subprocess.Popen`.



- `open_file(file_path: str)`: Classified as `MEDIUM` risk (requires user confirmation). Verifies that the path is inside an indexed folder, then launches the file in its default Windows viewer via `os.startfile()`.
- `lock_system()`: Classified as `HIGH` risk (requires user confirmation). Calls `ctypes.windll.user32.LockWorkStation()` to lock the Windows session.

### tools/\_\_\_init\_\_\_py

Path: tools/\_\_\_init\_\_\_py • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `tools` module.

## 18.8 Local RAG Subsystem (rag/)

### rag/ingestion.py

Path: rag/ingestion.py • Size: ~8,985 bytes • Lines: 232

**What it does:** Implements the complete document indexing pipeline: folder crawling, SHA-256 change detection, parser dispatch, chunking, embedding generation, and vector/FTS storage.

#### Code Explanation:

- `IngestionPipeline`: Coordinates indexing jobs. Initialized with DB connection, embedding engine, vector store, and chunker.
- `index_folder(folder_path, progress_callback=None)`: Recursively crawls `folder_path`. For each file matching supported extensions:
  1. Computes SHA-256 hash. If hash matches existing record in `documents` table, skips processing immediately.
  2. Dispatches file to appropriate parser in `documents.registry`.
  3. Passes extracted segments to `Chunker.chunk_document()`.
  4. Batch generates 384-dim embeddings via `EmbeddingsEngine.embed_batch()`.
  5. Inserts text chunks into SQLite `document_chunks` (which automatically populates `document_chunks_fts` via triggers).
  6. Inserts dense vectors into FAISS `VectorStore` and saves index.
- `retry_failed()`: Re-attempts indexing on files marked with status `failed`.

### rag/hybrid\_retriever.py

Path: rag/hybrid\_retriever.py • Size: ~7,802 bytes • Lines: 200

**What it does:** Executes hybrid keyword (BM25) and dense vector search, score normalization, weighted fusion, and cross-encoder reranking.

#### Code Explanation:

- `RetrievedChunk(dataclass)`: Encapsulates chunk ID, document ID, file path, page number, section, text, and composite relevance score.
- `HybridRetriever`: Initialized with keyword store, vector store, embeddings engine, and optional cross-encoder reranker.
- `retrieve(query: str, top_k: int = 5) -> list[RetrievedChunk]`:
  1. Retrieves candidate chunk IDs and BM25 scores from SQLite FTS5 via `KeywordStore.search()`.
  2. Generates query embedding and retrieves candidate chunk IDs and L2 distances from FAISS via `VectorStore.search()`.
  3. Converts L2 distances to similarities and applies min-max normalization to scale both BM25 and vector scores to `[0.0, 1.0]`.
  4. Calculates blended score: `score = (0.5 * bm25_norm) + (0.5 * dense_norm)`.
  5. If reranker is enabled, passes top 15 candidate chunks through `CrossEncoderReranker.rerank()`.
  6. Returns top `top_k` highest-scoring chunks.

### rag/chunking.py

Path: rag/chunking.py • Size: ~3,410 bytes • Lines: 104

**What it does:** Splits extracted document text segments into uniform chunks with overlapping token boundaries.

#### Code Explanation:

- **Chunker:** Uses sliding window tokenization with `chunk_size = 650` tokens and `chunk_overlap = 75` tokens.
- Preserves paragraph and sentence boundaries where possible to prevent semantic splitting.
- Attaches source metadata (page numbers, section headings) to each produced chunk.

### rag/embeddings.py

Path: rag/embeddings.py • Size: ~2,890 bytes • Lines: 86

**What it does:** Generates 384-dimensional dense vector embeddings locally using the `sentence-transformers` library.

#### Code Explanation:

- **EmbeddingsEngine:** Loads `all-MiniLM-L6-v2` onto CPU.
- `embed_text(text: str) -> np.ndarray`: Encodes a single string into a normalized 384-float32 vector.
- `embed_batch(texts: list[str], batch_size: int = 32) -> np.ndarray`: Encodes multiple chunks in batches for optimal CPU throughput.

### rag/vector\_store.py

Path: rag/vector\_store.py • Size: ~4,620 bytes • Lines: 142

**What it does:** Manages the FAISS CPU vector index, handling vector insertion, similarity search, chunk removal, and disk persistence.

#### Code Explanation:

- **VectorStore:** Wraps `faiss.IndexFlatL2(384)`. Maintains an in-memory `chunk_ids: list[int]` array mapping vector index positions to SQLite chunk primary keys.
- `add(chunk_ids, vectors)`: Adds normalized float32 vectors to the index.
- `search(query_vector, top_k=20) -> list[tuple[int, float]]`: Executes nearest-neighbor search, returning matched chunk IDs and L2 distances.
- `remove_chunks(chunk_ids)`: Rebuilds index excluding deleted chunks.
- `save(index_path) / load(index_path)`: Serializes index via `faiss.write_index` and deserializes via `faiss.read_index`.

### rag/keyword\_store.py

Path: rag/keyword\_store.py • Size: ~2,480 bytes • Lines: 76

**What it does:** Manages BM25 keyword retrieval through SQLite's built-in FTS5 engine.

#### Code Explanation:

- **KeywordStore:** Queries the `document_chunks_fts` virtual table.
- `search(query: str, top_k: int = 20) -> list[tuple[int, float]]`: Sanitizes query string (escaping FTS5 operators) and executes `SELECT rowid, bm25(document_chunks_fts) FROM document_chunks_fts WHERE document_chunks_fts MATCH ? ORDER BY bm25(...) LIMIT ?`.

### rag/reranker.py

Path: rag/reranker.py • Size: ~3,120 bytes • Lines: 92

**What it does:** Performs deep cross-attention sequence classification on top retrieval candidates to elevate precision.

#### Code Explanation:

- **CrossEncoderReranker:** Wraps `sentence-transformers.CrossEncoder` running `cross-encoder/ms-marco-MiniLM-L-6-v2` locally on CPU.
- `rerank(query: str, chunks: list[RetrievedChunk], top_k: int = 5) -> list[RetrievedChunk]`: Pairs

the query with each chunk's text (`[[query, chunk.text], ...]`), computes cross-attention relevance logits, sorts chunks by descending score, and returns the top `top_k` results.

### rag/indexing\_state.py

Path: `rag/indexing_state.py` • Size: ~2,150 bytes • Lines: 68

**What it does:** Thread-safe state tracker for monitoring active folder indexing jobs in the background.

#### Code Explanation:

- `IndexingState`: Tracks `is_indexing`: `bool`, `total_files`: `int`, `processed_files`: `int`, `current_file`: `str`, and error lists. Uses a `threading.Lock` to guarantee safe updates from background worker threads and safe reads from the UI progress bar.

### rag/\_\_init\_\_.py

Path: `rag/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `rag` module.

## 18.9 Web Layer (`web/`)

### web/search.py

Path: `web/search.py` • Size: ~2,992 bytes • Lines: 90

**What it does:** Asynchronous client connecting to SearXNG's JSON API to perform privacy-respecting search queries.

#### Code Explanation:

- `SearchResult` (dataclass): Encapsulates `title`, `url`, `snippet`, and `engine`.
- `SearXNGUnavailableError`: Raised when the endpoint is unreachable.
- `SearXNGClient`: Connects to `http://localhost:8080/search?format=json&q=...` using `httpx.AsyncClient` with a 10-second timeout. Parses the `results` array, filters tracking parameters, and returns typed `SearchResult` objects.

### web/crawler.py

Path: `web/crawler.py` • Size: ~2,474 bytes • Lines: 83

**What it does:** Fetches remote web pages with in-memory TTL caching, content-type verification, and response size guards.

#### Code Explanation:

- `fetch_html(url: str, client: httpx.AsyncClient = None) -> str`:
  1. Checks an in-memory dictionary cache with a 1-hour expiration timestamp. If cached and valid, returns cached HTML immediately.
  2. Issues an HTTP `GET` request with a custom descriptive User-Agent.
  3. Inspects `Content-Type` header (must include `text/html`).
  4. Checks `Content-Length` and streams body up to a strict 5 MB limit. If exceeded, truncates download to prevent memory exhaustion.
  5. Caches and returns raw HTML string.

### web/extraction.py

Path: `web/extraction.py` • Size: ~1,982 bytes • Lines: 63

**What it does:** Extracts clean article text from raw HTML using Mozilla's Readability algorithm, falling back to BeautifulSoup.

#### Code Explanation:

- `extract_readable_text(html: str) -> tuple[str, str]` (returns title, body):

1. Attempts extraction via `readability-lxml` (`Document(html).summary()`), which strips ads, sidebars, headers, and navigation menus.
2. If `readability-lxml` fails or produces sparse output, falls back to BeautifulSoup: removes `<script>`, `<style>`, `<nav>`, `<footer>` tags and extracts text.
3. Collapses whitespace and caps total extracted characters at `DEFAULT_EXTRACTION_MAX_CHARS` (20,000 characters).

### web/searxng\_process.py

Path: `web/searxng_process.py` • Size: ~6,892 bytes • Lines: 180

**What it does:** Manages the lifecycle of a local SearXNG child process, providing lazy startup and clean shutdown.

#### Code Explanation:

- `SearXNGProcessManager`: Singleton controller.
  - `is_running()` -> bool: Pings `http://localhost:8080` to determine if an external or managed instance is already healthy.
  - `ensure_running()`: Called lazily when `web_search` is invoked. If not running, launches SearXNG via `subprocess.Popen` and polls for health up to `DEFAULT_SEARXNG_STARTUP_WAIT_SECONDS` (15s).
  - `stop()`: Terminates the managed child process during application shutdown.

### web/\_\_init\_\_.py

Path: `web/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `web` module.

## 18.10 Document Parsers (`documents/`)

### documents/base.py

Path: `documents/base.py` • Size: ~1,850 bytes • Lines: 58

**What it does:** Defines the common extraction protocol, segment data structures, and parser exception types.

#### Code Explanation:

- `Segment` (dataclass): Contains `text: str`, `page_number: int | None`, and `section_name: str | None`.
- `ExtractedDocument` (dataclass): Contains `file_path: Path`, `segments: list[Segment]`, and `source_type: str`.
- `DocumentParser` (Protocol): Interface requiring `def parse(self, path: Path) -> ExtractedDocument`.
- `DocumentParseError` (Exception): Standardized exception raised when a file is corrupted, encrypted, or malformed.

### documents/registry.py

Path: `documents/registry.py` • Size: ~2,120 bytes • Lines: 65

**What it does:** Maps file extensions to their corresponding parser implementations.

#### Code Explanation:

- Maintains internal dictionary mapping extensions (e.g., `.pdf` → `PDFParser`, `.docx` → `DocxParser`, `.txt` → `TextParser`).
- `get_parser(path: Path) -> DocumentParser | None`: Inspects suffix and returns appropriate parser instance.
- `is_supported(path: Path) -> bool`: Returns whether a file extension has a registered parser.

### documents/pdf.py

Path: `documents/pdf.py` • Size: ~3,150 bytes • Lines: 95

**What it does:** Extracts text and table data from PDF files using PyMuPDF (fitz).

**Code Explanation:**

- `PDFParser(DocumentParser)`: Opens PDF via `pymupdf.open(path)`.
- Iterates through pages, extracting text blocks via `page.get_text("blocks")`.
- Packages each page's content as a distinct `Segment` tagged with `page_number = i + 1`. Handles encrypted/password-protected PDFs gracefully.

**documents/docx.py**

Path: `documents/docx.py` • Size: ~2,840 bytes • Lines: 86

**What it does:** Extracts paragraphs and tables from Microsoft Word (.docx) documents using `python-docx`.

**Code Explanation:**

- `DocxParser(DocumentParser)`: Opens document via `docx.Document(path)`.
- Extracts heading paragraph styles (Heading 1, Heading 2) to establish `section_name` boundaries.
- Iterates through document tables, formatting rows into clean tab-delimited text blocks.

**documents/pptx.py**

Path: `documents/pptx.py` • Size: ~2,950 bytes • Lines: 88

**What it does:** Extracts text and speaker notes from PowerPoint (.pptx) presentations using `python-pptx`.

**Code Explanation:**

- `PptxParser(DocumentParser)`: Opens presentation via `pptx.Presentation(path)`.
- Iterates through slides: extracts text from title shapes, body shapes, and slide speaker notes.
- Creates slide-level `Segment` objects tagged with slide numbers.

**documents/xlsx.py**

Path: `documents/xlsx.py` • Size: ~3,200 bytes • Lines: 98

**What it does:** Extracts structured row data from Excel (.xlsx) workbooks using `openpyxl`.

**Code Explanation:**

- `XlsxParser(DocumentParser)`: Opens workbook in `read_only=True` mode for minimal memory footprint.
- Iterates through worksheets: converts non-empty rows into tab-separated values.
- Packages each sheet as a `Segment` tagged with the sheet name as its section identifier.

**documents/text.py**

Path: `documents/text.py` • Size: ~3,450 bytes • Lines: 105

**What it does:** Universal parser for plain text, Markdown, CSV, JSON, HTML, and programming language source files.

**Code Explanation:**

- `TextParser(DocumentParser)`: Handles `.txt`, `.md`, `.csv`, `.json`, `.py`, `.js`, `.html`, etc.
- Reads file using UTF-8 with automatic fallback to Latin-1 encoding.
- For HTML files: strips tags using BeautifulSoup (`soup.get_text()`).
- Returns the entire content as a structured `Segment`.

**documents/\_\_init\_\_.py**

Path: `documents/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `documents` module.

## 18.11 Background Scheduler (`scheduler/`)

**scheduler/service.py**

Path: `scheduler/service.py` • Size: ~3,820 bytes • Lines: 114

**What it does:** Wraps APScheduler's `BackgroundScheduler` to poll SQLite for due reminders and dispatch Qt notifications.

#### Code Explanation:

- `SchedulerService(QObject)`: Inherits from Qt `QObject` to provide thread-safe signals: `reminder_due = Signal(dict)`.
- `start()`: Configures a background interval job firing every 30 seconds.
- In each execution cycle, calls `scheduler.reminders.check_and_fire_reminders()`. For each fired reminder, emits `reminder_due.emit(reminder_data)` across the thread boundary to the GUI.
- `shutdown()`: Stops the APScheduler background thread cleanly.

#### `scheduler/reminders.py`

Path: `scheduler/reminders.py` • Size: ~2,450 bytes • Lines: 75

**What it does:** Contains the polling logic that queries SQLite for pending reminders past their due time.

#### Code Explanation:

- `check_and_fire_reminders(conn) -> list[dict]`:
  1. Retrieves current localized timestamp using `JARVIS_TIMEZONE`.
  2. Calls `storage.repositories.reminders.get_due_reminders()`.
  3. Updates reminder state to `FIRE`D in the database.
  4. Returns list of triggered reminder dictionaries.

#### `scheduler/jobs.py`

Path: `scheduler/jobs.py` • Size: ~1,850 bytes • Lines: 56

**What it does:** Job execution wrappers for recurring background tasks.

#### Code Explanation:

- Wraps scheduled job executions in comprehensive error handling, logging exceptions to prevent APScheduler thread crashes.

#### `scheduler/__init__.py`

Path: `scheduler/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `scheduler` module.

## 18.12 Security & Audit (`security/`)

#### `security/audit.py`

Path: `security/audit.py` • Size: ~4,496 bytes • Lines: 148

**What it does:** Implements non-raising security auditing, writing operational logs and timing metrics to the `audit_log` table.

#### Code Explanation:

- `log_event(conn, event_type, action, status="SUCCESS", details=None, duration_ms=0)`: Inserts a row into `audit_log`. Passes details through `scrub_secrets()` before serialization. Catches and swallows all DB exceptions to prevent audit failures from impacting user workflows.
- `audit_action(conn, event_type, action, details=None)`: Context manager that records start time via `time.perf_counter()`, yields execution, records duration, and writes an audit event upon exit.

#### `security/permissions.py`

Path: `security/permissions.py` • Size: ~1,389 bytes • Lines: 38

**What it does:** Evaluates tool execution permissions based on risk tiers and confirmation flags.

**Code Explanation:**

- `PermissionDecision(dataclass)`: Contains `allowed: bool`, `requires_confirmation: bool`, and `reason: str`.
- `check_permission(tool: Tool) -> PermissionDecision`: If `tool.metadata.requires_confirmation` is `True` or `risk_level == HIGH`, returns decision with `requires_confirmation = True`. Otherwise permits immediate execution.

### security/secrets.py

Path: `security/secrets.py` • Size: ~1,874 bytes • Lines: 61

**What it does:** Sanitizes text strings and logs, masking secret keys to prevent accidental credential leakage.

**Code Explanation:**

- `mask(secret: str) -> str`: Replaces middle characters with ellipsis, retaining only the first 4 and last 2 characters (e.g., `"gsk_abcdef1234" → "gsk_...34"`).
- `scrub_secrets(text: str, known_secrets: list[str]) -> str`: Performs regex search-and-replace across arbitrary text, masking any occurrence of known API keys or credentials.
- `known_secrets_from_settings(settings) -> list[str]`: Extracts raw secret strings from configured Settings fields for scrubbing.

### security/\_\_init\_\_.py

Path: `security/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `security` module.

## 18.13 User Interface Subsystem (ui/)

### ui/chat\_window.py

Path: `ui/chat_window.py` • Size: ~29,652 bytes • Lines: 701

**What it does:** The primary desktop interface. Implements the chat popup, presence dot animation, push-to-talk mic button, confirmation modal, and background QThread workers.

**Code Explanation:**

- Worker QThreads:
  - `ChatTurnWorker(QThread)`: Runs agent turn asynchronously on a background thread. Emits `succeeded(dict)`, `failed(str)`, `status_updated(str)`.
  - `ConfirmTurnWorker(QThread)`: Deterministically executes a confirmed HIGH-risk tool call off the main thread.
  - `VoiceTurnWorker(QThread)`: Coordinates audio transcription, agent turn, TTS audio synthesis, and playback.
- `ChatWindow(QMainWindow)`:
  - Custom title bar with presence dot, logo, and kebab menu.
  - Embedded `ChatLog(QTextBrowser)` for message rendering.
  - Auto-resizing text input box and push-to-talk microphone button.
  - `_on_turn_succeeded()`: Renders assistant response using `render_markdown()` and attaches citation chips.
  - `_prompt_confirmation(request)`: Pops up a confirmation dialog for HIGH-risk actions.
  - `_animate_presence()`: Animates presence dot (idle, thinking, listening, speaking).
- `run(health_warnings=None)`: Application initialization entry point for the GUI.

### ui/messages.py

Path: `ui/messages.py` • Size: ~10,522 bytes • Lines: 219

**What it does:** Custom `QTextBrowser` that formats user and assistant messages into aligned, styled HTML message bubbles with interactive citation chips.



**Code Explanation:**

- `ChatLog(QTextBrowser):`
  - `append_user_message(text):` Generates a right-aligned teal message bubble.
  - `append_assistant_message(html_content, citations):` Generates a left-aligned gray bubble with rendered Markdown HTML.
  - `_render_citations(citations):` Appends clickable chips below assistant responses.
  - `_handle_anchor_click(url):` Intercepts link clicks: opens file paths in Windows shell and web URLs in default browser.

**ui/markdown\_render.py**

Path: ui/markdown\_render.py • Size: ~1,850 bytes • Lines: 56

**What it does:** Converts LLM-generated Markdown text into formatted HTML for rendering inside Qt message bubbles.**Code Explanation:**

- `render_markdown(text: str) -> str:`
  - Uses the Python markdown library with extensions: `tables`, `fenced_code`, `nl2br`, and `sane_lists`.
  - Ensures code blocks, bold text, lists, and tables render with clean typography.
  - Includes graceful fallback to plain HTML entity escaping if the markdown library is missing.

**ui/theme.py**

Path: ui/theme.py • Size: ~4,120 bytes • Lines: 128

**What it does:** Defines the application theme system with 14 color tokens, supporting light mode, dark mode, and Windows system auto-detection.**Code Explanation:**

- `Theme(dataclass):` Declares 14 color tokens: `bg_window`, `bg_input`, `bubble_user`, `bubble_assistant`, `text_primary`, `accent_teal`, etc.
- `LIGHT_THEME / DARK_THEME:` Predefined palettes matching JARVIS design specs.
- `detect_system_theme():` Uses Qt's `QGuiApplication.styleHints().colorScheme()` to query the Windows 10/11 dark mode setting.
- `generate_stylesheet(theme):` Returns complete Qt CSS string for window styling.

**ui/icon.py**

Path: ui/icon.py • Size: ~2,450 bytes • Lines: 75

**What it does:** Programmatically generates the JARVIS application logo as an SVG and rasterizes it to a `QIcon`.**Code Explanation:**

- `get_app_icon() -> QIcon:` Generates an SVG string representing the JARVIS geometric teal arc design, renders it into a `QPixmap`, and returns a high-resolution `QIcon` for the window and system tray.

**ui/tray.py**

Path: ui/tray.py • Size: ~3,650 bytes • Lines: 112

**What it does:** Implements the system tray icon and context menu for quick desktop access.**Code Explanation:**

- `JarvisTrayIcon(QSystemTrayIcon):` Creates tray icon with context menu options: "Open JARVIS", "Pause/Resume Indexing", "Voice Enabled (Toggle)", "Settings", and "Quit".
- Handles left-click activation to restore and focus the chat window.

**ui/hotkey.py**

Path: ui/hotkey.py • Size: ~2,120 bytes • Lines: 65



**What it does:** Registers the global hotkey (Ctrl+Space) using the `keyboard` library.

**Code Explanation:**

- `register_global_hotkey(hotkey_str, callback)`: Hooks into low-level Windows keyboard events via `keyboard.add_hotkey()`, invoking `callback` whenever the combination is pressed across any application.

### ui/settings.py

Path: `ui/settings.py` • Size: ~5,820 bytes • Lines: 168

**What it does:** Implements the Settings modal dialog for managing indexed folders and Windows startup options.

**Code Explanation:**

- `SettingsDialog(QDialog)`:
  - Folder Management tab: Lists currently indexed folders with checkboxes to enable/disable, and buttons to "Add Folder" (via `QFileDialog`) or "Remove".
  - Triggers background re-indexing when new folders are added.
  - Startup toggle: Checkbox linked to `app.windows_startup` to enable/disable "Start with Windows".

### ui/history.py

Path: `ui/history.py` • Size: ~4,250 bytes • Lines: 130

**What it does:** Modal dialog for browsing and switching between past conversation sessions.

**Code Explanation:**

- `HistoryDialog(QDialog)`: Displays a list of past conversations with titles and last-updated timestamps. Selecting an item switches the active conversation in the main chat window.

### ui/notifications.py

Path: `ui/notifications.py` • Size: ~2,350 bytes • Lines: 72

**What it does:** Dispatches native Windows toast notifications for reminders and system alerts.

**Code Explanation:**

- `NotificationService`: Uses Qt's `QSystemTrayIcon.showMessage()` to display native Windows notification banners with title, message body, and application icon.

### ui/usage\_dialog.py

Path: `ui/usage_dialog.py` • Size: ~4,150 bytes • Lines: 125

**What it does:** Displays token consumption and cost analytics dialog.

**Code Explanation:**

- `UsageDialog(QDialog)`: Queries `storage.repositories.usage` and `voice_usage`. Renders summary tables showing session and all-time prompt tokens, completion tokens, voice audio seconds, and total estimated expenditure in USD.

### ui/app.py

Path: `ui/app.py` • Size: ~1,650 bytes • Lines: 52

**What it does:** Helper module containing the main application execution loop.

**Code Explanation:**

- Wraps `QApplication` creation, theme resolution, and window presentation.

### ui/\_\_init\_\_.py

Path: `ui/__init__.py` • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `ui` module.

## 18.14 Voice Subsystem (`voice/`)

### voice/audio.py

Path: `voice/audio.py` • Size: ~3,287 bytes • Lines: 98

**What it does:** Handles audio input recording via PortAudio/sounddevice, RMS silence detection gating, and audio output playback.

#### Code Explanation:

- `AudioRecorder`:
  - `start()`: Opens a 16,000 Hz, 16-bit mono input stream using `sounddevice.InputStream`. Appends incoming float32 sample blocks into an in-memory queue.
  - `stop()` -> `np.ndarray`: Stops the input stream, concatenates all recorded chunks into a single NumPy array, and returns the raw audio data.
- `is_silent(audio: np.ndarray, sample_rate=16000) -> bool`:
  1. Checks duration: if audio is shorter than `DEFAULT_SILENCE_MIN_SECONDS` (0.5s), returns `True`.
  2. Computes Root-Mean-Square (RMS) energy: `rms = np.sqrt(np.mean(audio**2))`. If `rms < DEFAULT_SILENCE_RMS_THRESHOLD` (0.01), returns `True`. Discards silent audio before invoking Whisper models.
- `play_audio(audio_data: bytes, sample_rate=22050)`: Plays synthesized TTS audio through the default Windows playback device using `sounddevice.play()`.

### voice/pricing.py

Path: `voice/pricing.py` • Size: ~2,582 bytes • Lines: 82

**What it does:** Maintains voice rate cards and computes USD billing for STT audio seconds and TTS characters.

#### Code Explanation:

- `calculate_stt_cost(provider, model, seconds) -> float`: Calculates cost for cloud STT (Groq Whisper, Deepgram Nova-3). Local faster-whisper is always \$0.00.
- `calculate_tts_cost(provider, model, character_count) -> float`: Calculates cost for cloud TTS (Groq PlayAI, Deepgram Aura). Local Piper TTS is always \$0.00.

### voice/stt.py

Path: `voice/stt.py` • Size: ~3,450 bytes • Lines: 108

**What it does:** Orchestrates the Speech-to-Text provider fallback chain (Groq → Deepgram → local faster-whisper).

#### Code Explanation:

- `transcribe(audio: np.ndarray, sample_rate=16000) -> str`:
  1. Checks silence gate: if `is_silent()` is `True`, returns empty string immediately.
  2. Iterates through configured providers: attempts Groq Whisper first; on network/rate-limit error, falls back to Deepgram Nova-3; on failure, falls back to local faster-whisper.
  3. Records duration and cost to `voice_usage` table.
  4. Returns transcribed text.

### voice/stt\_groq.py

Path: `voice/stt_groq.py` • Size: ~2,750 bytes • Lines: 85

**What it does:** Cloud STT backend using Groq's hosted Whisper Large v3 Turbo API.

#### Code Explanation:

- Converts NumPy float32 audio into WAV byte buffer using Python `wave` module.
- Sends multipart POST to `https://api.groq.com/openai/v1/audio/transcriptions` with model `whisper-large-v3-turbo`.
- Parses and returns transcription text.

### voice/stt\_deepgram.py

Path: voice/stt\_deepgram.py • Size: ~2,890 bytes • Lines: 88

**What it does:** Cloud STT backend using Deepgram's hosted Listen API with model `nova-3`.

#### Code Explanation:

- Sends raw WAV audio via POST to `https://api.deepgram.com/v1/listen?model=nova-3&smart_format=true`.
- Extracts transcript from Deepgram's JSON response hierarchy.

### voice/stt\_local.py

Path: voice/stt\_local.py • Size: ~3,120 bytes • Lines: 95

**What it does:** 100% offline, local STT backend using `faster-whisper` running on CPU with 8-bit quantization.

#### Code Explanation:

- Initializes `faster_whisper.WhisperModel("base", device="cpu", compute_type="int8")`.
- Executes `model.transcribe(audio, beam_size=5)`.
- Concatenates transcribed segments into a final string with zero cloud dependencies.

### voice/tts.py

Path: voice/tts.py • Size: ~3,520 bytes • Lines: 110

**What it does:** Orchestrates the Text-to-Speech provider fallback chain (Groq → Deepgram → local Piper).

#### Code Explanation:

- `synthesize(text: str) -> bytes:`
  1. Iterates through configured TTS providers: attempts Groq PlayAI first; on failure, falls back to Deepgram Aura; on failure, falls back to local Piper TTS.
  2. Records character count and billing cost to `voice_usage` table.
  3. Returns raw audio bytes for playback.

### voice/tts\_groq.py

Path: voice/tts\_groq.py • Size: ~2,680 bytes • Lines: 82

**What it does:** Cloud TTS backend using Groq's PlayAI Orpheus voice synthesis endpoint.

#### Code Explanation:

- Sends POST to Groq's audio speech endpoint with requested text and voice model. Returns audio byte buffer.

### voice/tts\_deepgram.py

Path: voice/tts\_deepgram.py • Size: ~2,750 bytes • Lines: 84

**What it does:** Cloud TTS backend using Deepgram's hosted Speak API with voice model `aura-asteria-en`.

#### Code Explanation:

- Sends POST to `https://api.deepgram.com/v1/speak?model=aura-asteria-en`. Returns synthesized MP3/WAV audio stream.

### voice/tts\_local.py

Path: voice/tts\_local.py • Size: ~3,250 bytes • Lines: 98

**What it does:** 100% offline, local TTS backend using `piper-tts` running ONNX models on CPU.

#### Code Explanation:

- Loads Piper voice model (`en_US-lessac-medium.onnx`) from `data/voices/`.
- Synthesizes natural-sounding speech locally with zero latency or cloud calls, returning raw PCM audio.

### voice/ \_\_init\_\_.py

Path: voice/ \_\_init\_\_.py • Size: ~0 bytes • Lines: 0

**What it does:** Package marker file for the `voice` module.

## 18.15 Project Root & Configuration Files

### start\_jarvis.bat

Path: start\_jarvis.bat • Size: ~280 bytes

**What it does:** Windows batch launcher for the main application. Activates the Python virtual environment (`.venv\Scripts\activate.bat`) and runs `python -m app.main`.

### start\_jarvis\_supervisor.bat

Path: start\_jarvis\_supervisor.bat • Size: ~290 bytes

**What it does:** Windows batch launcher for the lightweight supervisor daemon. Activates the virtual environment and runs `python -m app.supervisor`.

### .env / .env.example

Path: .env, .env.example • Size: ~1,850 bytes

**What it does:** Environment configuration files. `.env` stores actual user secrets and API keys (never committed to version control). `.env.example` provides a fully documented template with default settings.

### requirements.txt

Path: requirements.txt • Size: ~1,248 bytes • Lines: 46

**What it does:** Itemizes all third-party Python package dependencies organized by development phase (core, LLM, RAG, scheduling, web, voice, Windows integration, and testing tools).

### pyproject.toml / spec.md / README.md

Path: pyproject.toml, spec.md, README.md

**What it does:** `pyproject.toml` defines project metadata and tooling configurations (Ruff, Mypy, Pytest). `spec.md` documents technical specifications and architectural contracts. `README.md` provides setup and usage instructions.